



KAPITEL

14

Fehlerbehandlung

Lernziele

Zentrale Konzepte in diesem Kapitel: defensive Programmierung, Fehler melden, Exceptions auslösen und behandeln, einfache Dateiverarbeitung

Java-Konstrukte in diesem Kapitel: `TreeMap`, `TreeSet`, `SortedMap`, `assert`, `Exception`, `throw`, `throws`, `try`, `catch`, `File`, `FileReader`, `FileWriter`, `Path`, `Scanner`, `Stream`

In Kapitel 9 haben wir erwähnt, dass logische Programmfehler schwieriger zu finden sind als syntaktische Fehler, weil ein Compiler bei logischen Fehlern nicht helfen kann. Logische Fehler entstehen aus unterschiedlichen Gründen, die sich oft überschneiden:

- Die Lösung für ein Problem wurde nicht korrekt implementiert. Beispielsweise könnte bei einer Statistikaufgabe für eine Menge an Werten statt des Medians (des „mittleren“ Werts) fälschlicherweise der Durchschnittswert berechnet werden.
- Ein Objekt könnte um eine Dienstleistung gebeten werden, die es nicht erbringen kann. Beispielsweise könnte die `get`-Methode eines Sammlungsobjekts mit einem Index aufgerufen werden, der außerhalb des gültigen Bereichs liegt.
- Ein Objekt könnte auf eine Weise benutzt werden, die der Entwerfer der Klasse so nicht vorhergesehen hat und die das Objekt in einen inkonsistenten oder unangemessenen Zustand versetzt. Das kann leicht passieren, wenn eine Klasse in einem anderen als ihrem ursprünglich geplanten Zusammenhang wiederverwendet wird, beispielsweise durch Vererbung.

Obwohl wir mit den in Kapitel 9 diskutierten Teststrategien viele logische Fehler finden und beheben können, bevor ein Programm zum Einsatz kommt, zeigt die Erfahrung doch auch, dass es weiterhin zu Programmfehlern kommen wird. Selbst das bestgetestete Programm kann scheitern, wenn Umstände zum Tragen kommen, die außerhalb des Einflussbereichs des Entwicklers liegen. Nehmen Sie den Fall eines Webbrowsers, der eine nicht existierende Seite anzeigen soll, oder eines Programms, das Daten in ein Dateisystem schreiben soll, dessen Speicher ausgeschöpft ist. Solche Probleme sind nicht die Folge logischer Programmierfehler, können aber ein Programm leicht scheitern lassen, wenn dieser Problemfall nicht vorhergesehen wurde.

In diesem Kapitel werden wir betrachten, wie Fehlersituationen vorausgesehen werden können und wie im laufenden Programm auf solche Fehlersituationen reagiert werden kann. Außerdem werden wir aufzeigen, wie auftretende Fehler gemeldet

werden können. Wir geben auch eine kurze Einführung in den Umgang mit Textdateien, da bei der Benutzung von Dateien sehr leicht Fehler auftreten können.

14.1 Das Adressbuch-Projekt

Wir werden eine Reihe von *Adressbuch*-Projekten benutzen, um die Prinzipien der Meldung von Fehlern und der Fehlerbehandlung zu demonstrieren, wie sie in vielen Anwendungen zum Einsatz kommen. Die Projekte modellieren eine Anwendung zur Verwaltung persönlicher Kontakte – Name, Adresse und Telefonnummer – für eine beliebige Anzahl an Personen. Die Kontakte werden im Adressbuch sowohl nach den Namen als auch nach den Telefonnummern indiziert. Die zentralen Klassen, die wir diskutieren werden, sind **Adressbuch** (Listing 14.1) und **Kontakt**. Zusätzlich ist eine Klasse **AdressbuchDemo** vorgegeben, die ein Adressbuch mit einigen Testdaten initialisiert.

Listing 14.1
Die Klasse
Adressbuch.

```
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Set;
import java.util.SortedMap;
import java.util.TreeMap;
import java.util.TreeSet;

/**
 * Eine Klasse zur Verwaltung einer beliebigen Anzahl von
 * Kontakten. Die Daten werden sowohl über den Namen
 * als auch über die Telefonnummer indiziert.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class Adressbuch
{
    // Speicher für eine beliebige Anzahl von Kontakten.
    private TreeMap<String, Kontakt> buch;
    private int anzahlEintraege;

    /**
     * Initialisiere ein neues Adressbuch.
     */
    public Adressbuch()
    {
        buch = new TreeMap<>();
        anzahlEintraege = 0;
    }

    /**
     * Schlage einen Namen oder eine Telefonnummer
     * nach und liefere den zugehörigen Kontakt.
     * @param schluessel der Name oder die Nummer zum Nachschlagen
     * @return den zum Schlüssel gehörenden Kontakt
     */
    public Kontakt gibKontakt(String schluessel)
    {
        return buch.get(schluessel);
    }

    /**
     * Ist der gegebene Schlüssel in diesem Adressbuch bekannt?
     * @param schluessel der Name oder die Nummer zum Nachschlagen
     * @return true wenn der Schlüssel eingetragen ist, false sonst
     */
    public boolean schluesselBekannt(String schluessel)
    {
        return buch.containsKey(schluessel);
    }
}
```

```

/**
 * Füge einen neuen Kontakt in dieses Adressbuch ein.
 * @param kontakt der neue Kontakt
 */
public void neuerKontakt(Kontakt kontakt)
{
    buch.put(kontakt.gibName(), kontakt);
    buch.put(kontakt.gibTelefon(), kontakt);
    anzahlEintraege++;
}

/**
 * Ändere die Kontaktdaten, die bisher unter dem gegebenen
 * Schlüssel eingetragen waren.
 * @param alterSchluessel einer der verwendeten Schlüssel
 * @param daten           die neuen Kontaktdaten
 */
public void aendereKontakt(String alterSchluessel,
                             Kontakt daten)
{
    entferneKontakt(alterSchluessel);
    neuerKontakt(daten);
}

/**
 * Suche nach allen Kontakten mit einem Schlüssel, der mit dem
 * gegebenen Präfix beginnt.
 * @param präfix der Schlüsselpräfix, nach dem gesucht werden soll
 * @return ein Array mit den gefundenen Kontakten
 */
public Kontakt[] suche(String praefix)
{
    // Eine Liste für die Treffer anlegen.
    List<Kontakt> treffer = new LinkedList<>();
    // Finden von Schlüssel, die gleich dem oder größer als
    // der Präfix sind.
    SortedMap<String, Kontakt> rest = buch.tailMap(praefix);
    Iterator<String> it = rest.keySet().iterator();
    // Stoppen bei der ersten Nichtübereinstimmung.
    boolean sucheBeendet = false;
    while(!sucheBeendet && it.hasNext()) {
        String schluessel = it.next();
        if(schluessel.startsWith(praefix)) {
            treffer.add(buch.get(schluessel));
        }
        else {
            sucheBeendet = true;
        }
    }
    Kontakt[] ergebnisse = new Kontakt[treffer.size()];
    treffer.toArray(ergebnisse);
    return ergebnisse;
}

/**
 * Liefere die Anzahl der Einträge aktuell in diesem Adressbuch.
 * @return die Anzahl der Einträge
 */
public int gibAnzahlEintraege()
{
    return anzahlEintraege;
}

/**
 * Entferne den Eintrag mit dem gegebenen Schlüssel aus
 * diesem Adressbuch.
 * @param schluessel einer der Schlüssel des Eintrags,
 *                  der entfernt werden soll
 */
public void entferneKontakt(String schluessel)
{
    Kontakt kontakt = buch.get(schluessel);
    buch.remove(kontakt.gibName());
    buch.remove(kontakt.gibTelefon());
    anzahlEintraege--;
}

```



```

/**
 * Liefere alle Kontaktdaten, sortiert nach der
 * Sortierreihenfolge, die die Klasse Kontakt definiert.
 * @return die sortierten Kontaktdaten
 */
public String alleKontaktdaten()
{
    // Weil jeder Satz unter zwei Schlüsseln eingetragen ist,
    // muss ein Set mit den Kontakten gebildet werden. Dadurch
    // werden Duplikate entfernt.
    StringBuilder alleEintraege = new StringBuilder();
    Set<Kontakt> sortierteDaten = new TreeSet<>(buch.values());
    for(Kontakt kontakt : sortierteDaten) {
        alleEintraege.append(kontakt).append("\n\n");
    }
    return alleEintraege.toString();
}
}

```

Neue Kontakte können mit der Methode **neuerKontakt** in das Adressbuch eingetragen werden. Dabei wird angenommen, dass der Kontakt wirklich neu ist und nicht einen bereits bestehenden Eintrag ändern soll. Für eine Änderung steht die Methode **aendereKontakt** zur Verfügung, die einen alten Eintrag entfernt und durch einen neuen ersetzt. Das Adressbuch bietet zwei Möglichkeiten zum Abfragen von Kontakten: Die Methode **gibKontakt** nimmt einen Namen oder eine Telefonnummer als Suchschlüssel und liefert den passenden Kontakt dazu; die Methode **suche** liefert ein Array aller Kontakte, deren Name oder Telefonnummer mit einem gegebenen Präfix beginnt. Beispielsweise liefert der Suchbegriff **"04"** alle Einträge, deren Vorwahl mit diesen Ziffern beginnt.

Es sind zwei einführende Versionen des *Adressbuch*-Projekts zum Erkunden vorgegeben. Beide ermöglichen den Zugriff auf die Version von Adressbuch, die in Listing 14.1 angegeben ist. Das Projekt *Adressbuch-V1T* bietet eine textbasierte Schnittstelle, ähnlich zur Schnittstelle des *Zuul*-Projekts aus Kapitel 8. In dieser Version stehen Befehle zum Auflisten des Adressbuchinhalts, zum Durchsuchen und zum Eintragen eines neuen Kontakts zur Verfügung. Von der Schnittstelle vermutlich interessanter ist die Version *Adressbuch-V1G*, die eine einfache grafische Benutzungsschnittstelle bietet. Experimentieren Sie mit beiden Versionen, um die Möglichkeiten der Anwendung zu erkunden.

Übung 14.1 Öffnen Sie das Projekt *Adressbuch-V1G* in BlueJ und erzeugen Sie ein Objekt der Klasse **AdressbuchDemo**. Rufen Sie seine Methode **zeigeSchnittstelle** auf und interagieren Sie über die grafische Schnittstelle mit dem Test-Adressbuch.

Übung 14.2 Wiederholen Sie Ihre Experimente mit der textbasierten Schnittstelle des Projekts *Adressbuch-V1T*.

Übung 14.3 Untersuchen Sie die Implementierung der Klasse **Adressbuch** und beurteilen Sie, ob sie gut entworfen ist oder nicht. Sehen Sie Kritikpunkte an der vorliegenden Implementierung?

Übung 14.4 Die Klasse **Adressbuch** benutzt etliche Klassen aus dem Paket **java.util**; wenn Sie mit diesen noch nicht vertraut sind, dann sehen Sie sich die API-Dokumentation an, um Ihre Lücken zu schließen. Halten Sie die Verwendung so vieler unterschiedlicher Klassen für gerechtfertigt? Hätte eine **HashMap** anstelle der **TreeMap** verwendet werden können?

Wenn Sie sich nicht sicher sind, versuchen Sie, **TreeMap** in ein **HashMap** zu ändern, und schauen Sie, ob **HashMap** Ihnen ebenfalls alle benötigte Funktionalität bietet.

Übung 14.5 Verändern Sie die Klassen **Befehlswoerter** und **AdressbuchTexteingabe** im Projekt *Adressbuch-V17* so, dass auch ein interaktiver Zugriff auf die Methoden **gibKontakt** und **entferneKontakt** der Klasse **Adressbuch** möglich ist.

Übung 14.6 Die Klasse **Adressbuch** definiert ein Datenfeld, das die Anzahl der Einträge im Adressbuch hält. Hielten Sie es für angemessener, wenn dieser Wert aus der Anzahl der eindeutigen Einträge in der **TreeMap** berechnet würde? Können Sie sich beispielsweise Umstände vorstellen, unter denen die folgende Berechnung nicht den gleichen Wert liefern würde?

```
return buch.size() / 2;
```

Übung 14.7 Wie leicht ist es Ihrer Meinung nach, ein **String**-Datenfeld für eine E-Mail-Adresse zu der Klasse **Kontakt** hinzuzufügen und es dann als dritten Schlüssel in **Adressbuch** zu verwenden?

14.2 Defensive Programmierung

14.2.1 Interaktion zwischen Klient und Dienstleister

Ein **Adressbuch** ist ein typisches Dienstleistungsobjekt, das von sich aus keine Aktionen durchführt; all seine Aktivitäten werden durch Anfragen von Klienten angestoßen. Die Programmierer einer Dienstleisterklasse können mindestens zwei grundlegende Sichtweisen einnehmen:

- Sie können annehmen, dass alle Klienten wissen, was sie tun, und nur sinnvolle und wohlstrukturierte Anfragen stellen.
- Sie können von einer möglicherweise fehlerhaften Umgebung ausgehen, in der alles unternommen werden sollte, damit Klienten den Dienstleister nicht auf falsche Weise benutzen können.

Das sind offensichtlich zwei extreme Positionen. In der Praxis liegt die Wahrheit üblicherweise irgendwo in der Mitte: Die meisten Anfragen von Klienten sind sinnvoll, aber es gibt vereinzelte fehlerhafte Anfragen – entweder aufgrund logischer Programmierfehler oder wegen falscher Annahmen des Entwicklers, der den Klienten programmiert hat. Eine dritte Möglichkeit ist natürlich ein absichtlich feindlich agierender Klient, der versucht, dem Server zu schaden und seine Schwächen auszunutzen.

Ausgehend von diesen beiden unterschiedlichen Sichtweisen lassen sich Fragen wie die folgenden diskutieren:

- Wie weit sollte ein Dienstleister die Anfragen von Klienten überprüfen?
- Wie sollte ein Dienstleister Fehler an seine Klienten melden?
- Wie kann ein Klient mögliche Scheiterungsgründe für eine Anfrage vorhersehen?
- Wie sollte ein Klient mit dem Scheitern einer Anfrage umgehen?

Wenn wir die Klasse **Adressbuch** mit diesen Fragen im Hinterkopf betrachten, dann erkennen wir, dass sie vollständig im Vertrauen auf eine sinnvolle Benutzung durch Klienten implementiert wurde. **Übung 14.8** zeigt eine Benutzungssituation, in der dieses Vertrauen zu einem Fehler führt.

Übung 14.8 Erzeugen Sie im Projekt *Adressbuch-V1G* ein neues **Adressbuch**-Objekt auf der Objektleiste. Dieses enthält dann noch keine Einträge. Rufen Sie nun die Methode **entferneKontakt** mit einem beliebigen Schlüsselwert auf. Was passiert? Können Sie erklären, warum das passiert?

Übung 14.9 Die einfachste Antwort eines Programmierers auf eine Fehler-situation ist der Abbruch des Programms (also ein „Absturz“). Können Sie sich Situationen vorstellen, in denen ein schlichter Programmabbruch gefährlich sein kann?

Übung 14.10 Viele kommerziell vertriebene Programme enthalten Fehler, die nicht korrekt behandelt werden und die Software deshalb zum Abstürzen bringen. Ist das unvermeidlich? Ist es akzeptabel? Diskutieren Sie.

Das Problem in der Methode **entferneKontakt** liegt darin, dass dort angenommen wird, dass der gegebene Schlüssel ein gültiger Schlüssel innerhalb des Adressbuches ist. In der Methode wird der gegebene Schlüssel benutzt, um den zugehörigen Kontakt auszulesen:

```
Kontakt kontakt = buch.get(schluesel);
```

Wenn für den Schlüssel aber kein zugeordnetes Objekt existiert, dann enthält die Variable **kontakt** nun den Wert **null**. Das ist an sich noch kein Fehler; der Fehler entsteht durch folgende Anweisung, bei der wir davon ausgehen, dass **kontakt** sich auf ein gültiges Objekt bezieht.

```
buch.remove(kontakt.gibName());
```

Es ist ein Fehler, eine Methode an einer Variablen aufzurufen, die **null** enthält, und das Ergebnis ist immer ein Laufzeitfehler. BlueJ meldet dies als eine **NullPointerException** und hebt im Quelltext die Zeile hervor, in der der Fehler aufgetreten ist. Wir werden später in diesem Kapitel Exceptions noch ausführlich besprechen. An dieser Stelle können wir einfach anmerken, dass ein solcher Fehler in einer laufenden Anwendung zu einem Programmabbruch führt (d.h., diese unkontrolliert beendet), bevor das Programm seine Aufgabe erfüllt hat.

Hier besteht also ein Problem; aber wessen Problem ist es? Ist es ein Fehler des Klienten-Objekts, weil es die Methode mit einem fehlerhaften Parameter aufgerufen hat, oder ist es ein Fehler des Dienstleisters, der mit dieser Situation nicht angemessen umgegangen ist? Der Programmierer der Klienten-Klasse könnte sagen, dass die Dokumentation der Methode nichts darüber aussagt, dass der Schlüssel gültig sein muss. Andererseits könnte der Programmierer des Dienstleisters sagen, dass es offensichtlich falsch ist, einen Kontakt mit einem ungültigen Schlüssel entfernen zu wollen. Unsere Absicht in diesem Kapitel ist nicht, einen solchen Streit aufzulösen, sondern ihn gar nicht erst auftreten zu lassen. Wir beginnen mit der Fehlerbehandlung aus Sicht des Dienstleisters.

Übung 14.11 Speichern Sie eine Kopie eines der *Adressbuch-V1*-Projekte unter einem anderen Namen, um auf dieser Kopie weiterarbeiten zu können. Verändern Sie die Methode `entferneKontakt` so, dass es bei einem Aufruf mit einem ungültigen Schlüssel nicht mehr zu einer `NullPointerException` kommt. Wenn der Schlüssel ungültig ist, sollte die Methode einfach nichts tun.

Übung 14.12 Ist es notwendig, einen Aufruf von `entferneKontakt` mit einem falschen Schlüssel zu melden? Wenn ja: Wie würden Sie dies melden?

Übung 14.13 Gibt es andere Methoden in der Klasse `Adressbuch`, die gegenüber ähnlichen Fehlern anfällig sind? Wenn ja, dann korrigieren Sie auch diese in Ihrer Version des Projekts. Ist es in allen Fällen angemessen, dass die Methode bei einem falschen Parameter einfach nichts tut? Sollten die Fehler auf irgendeine Weise gemeldet werden? Wenn ja: Wie würden Sie dies tun und würde die Meldung für alle Fehler gleich aussehen?

14.2.2 Parameter prüfen

Ein Dienstleister ist am anfälligsten, wenn seine Konstruktoren oder Methoden Parameterwerte übergeben bekommen. Die an einen Konstruktor übergebenen Werte initialisieren üblicherweise den Objektzustand; die an eine Methode übergebenen Parameter beeinflussen die Auswirkungen des Methodenaufrufs und möglicherweise seinen Ergebniswert. Deshalb sollte ein Dienstleister sehr genau wissen, ob er den übergebenen Parametern blind vertraut oder ob er sie selbst auf ihre Gültigkeit überprüft. In der momentanen Version wird sowohl in der Klasse `Kontakt` als auch in der Klasse `Adressbuch` keinerlei Überprüfung der Parameter durchgeführt. Wie wir am Beispiel der Methode `entferneKontakt` gesehen haben, kann dies zu einem schweren Laufzeitfehler führen.

Es ist relativ einfach, eine `NullPointerException` in `entferneKontakt` zu verhindern; Listing 14.2 zeigt, wie das getan werden kann. Beachten Sie, dass wir neben der Implementierung auch den Kommentar der Methode geändert haben, damit Klienten wissen, dass unbekannte Schlüssel ignoriert werden.

Listing 14.2

Ignorieren unbekannter Schlüssel in `entferneKontakt`.

```
/**
 * Entferne den Eintrag mit dem gegebenen Schlüssel aus
 * diesem Adressbuch. Bei einem unbekannten Schlüssel tue nichts.
 * @param schlüssel einer der Schlüssel des Eintrags,
 *                der entfernt werden soll
 */
public void entferneKontakt(String schluesssel)
{
    if(schluessselBekannt(schluesssel)) {
        Kontakt kontakt = buch.get(schluesssel);
        buch.remove(kontakt.gibName());
        buch.remove(kontakt.gibTelefon());
        anzahlEintraege--;
    }
}
```

Wenn wir die anderen Methoden der Klasse **Adressbuch** untersuchen, dann fallen uns weitere Stellen für ähnliche Verbesserungen auf:

- Die Methode **neuerKontakt** sollte prüfen, dass ihr Parameter nicht den Wert **null** hat.
- Die Methode **aendereKontakt** sollte prüfen, dass der alte Schlüssel bekannt ist und der Parameter nicht den Wert **null** hat.
- Die Methode **suche** sollte prüfen, dass der Schlüssel nicht **null** ist.

Diese Änderungen wurden alle in der jeweils zweiten Version der bisher betrachteten Projekte umgesetzt: *Adressbuch-V2T* und *Adressbuch-V2G*.

Übung 14.14 Was denken Sie, warum wir ähnliche Änderungen nicht auch an den Methoden **gibKontakt** und **schluessselBekannt** vornehmen wollten?

Übung 14.15 Bei der Behandlung von Parameterfehlern haben wir keine Fehlermeldungen ausgegeben. Denken Sie, dass ein **Adressbuch** eine Fehlermeldung ausgeben *sollte*, wenn es einen fehlerhaften Parameter bekommt? Gibt es Situationen, in denen eine Fehlermeldung nicht angemessen wäre? Sind beispielsweise auf der Konsole ausgegebene Fehlermeldungen unangemessen bei der GUI-Version des Projekts?

Übung 14.16 Gibt es weitere Überprüfungen, die wir Ihrer Meinung nach an den Parametern der anderen Methoden vornehmen sollten, damit ein **Adressbuch** nicht in einen inkonsistenten Zustand kommen kann?

14.3 Fehlermeldungen durch den Dienstleister

Nachdem wir einen Dienstleister vor illegalen Operationen durch falsche Parameter bewahrt haben, könnten wir jetzt die Meinung vertreten, dass der Programmierer eines Dienstleisters nichts weiter zu tun braucht. Tatsächlich würden wir aber gerne solche Fehlersituationen von vornherein ausschließen. Außerdem sind falsche Parameter meist auf Programmierfehler im Klienten zurückzuführen. Deshalb sollten wir, statt nur uns selbst zu schützen, auch den Klienten auf einen

aufgetretenen Fehler hinweisen – entweder das benutzende Objekt selbst oder den Benutzer oder den Programmierer. Auf diese Weise gibt es eine reale Chance, dass der fehlerhafte Code korrigiert wird. Beachten Sie aber, dass wir es hier mit drei ganz unterschiedlichen Typen von „Adressaten“ für die Benachrichtigung zu tun haben.

Welches ist die beste Art für einen Dienstleister, ein aufgetretenes Problem zu melden? Es gibt nicht die eine richtige Antwort auf diese Frage, und die angemessene Antwort hängt häufig vom Zusammenhang ab, in dem der Dienstleister verwendet wird. In den folgenden Abschnitten werden wir einige Optionen für die Meldung von Fehlern untersuchen.

Übung 14.17 Auf wie viele Arten kann eine Methode inkorrekte Parameterwerte oder andere Umstände melden, die sie davon abhalten, ihre Aufgabe korrekt auszuführen? Betrachten Sie möglichst viele unterschiedliche Arten von Anwendungen. Beispielsweise solche mit einer grafischen Benutzerschnittstelle; solche mit einer textbasierten Schnittstelle und einem menschlichen Benutzer; aber auch solche, in denen keine Interaktion mit einem Benutzer stattfindet, etwa beim elektronischen Motorenmanagement eines Autos; oder Software in eingebetteten Systemen, wie z.B. ein Bankautomat.

14.3.1 Den Benutzer informieren

Die offensichtliche Möglichkeit, mit der ein Objekt auf einen fehlerhaften Umstand reagieren kann, ist, den Benutzer der Anwendung zu benachrichtigen. Diese Option bedeutet entweder die Ausgabe einer Fehlermeldung über **System.out** oder **System.err** oder das Öffnen eines Fensters mit einem Fehlertext.

Die Hauptprobleme mit beiden Ansätzen sind:

- Sie gehen davon aus, dass die Anwendung von einem Benutzer bedient wird, der die Fehlermeldung zu sehen bekommt. Es gibt aber viele Anwendungen, die völlig unabhängig von einem Benutzer ausgeführt werden. Eine Fehlermeldung oder ein Fehlerfenster bleibt dann völlig unbeachtet. Tatsächlich könnte der Computer, auf dem die Anwendung läuft, gar keinen Bildschirm zur Ausgabe zur Verfügung haben.
- Selbst wenn ein Benutzer die Fehlermeldung sehen kann, wird er selten in der Lage sein, etwas gegen das Problem zu unternehmen. Stellen Sie sich einen Bankautomaten vor, der Ihnen eine **NullPointerException** meldet! Nur in den Fällen, in denen eine unmittelbare Aktion des Benutzers zu dem Fehler geführt hat – etwa eine falsche Eingabe –, kann der Benutzer möglicherweise das nächste Mal korrigierend eingreifen.

Programme, die unangemessene Fehlermeldungen ausgeben, werden ihre Benutzer eher verärgern, als ihnen mit den Meldungen zu nützen. Deshalb ist, abgesehen von sehr wenigen Situationen, das Informieren des Benutzers keine allgemeingültige Lösung zur Frage der Fehlerbehandlung.

Übung 14.18 Die Java-API enthält im Paket `java.util.logging` eine durchdachte Sammlung von Klassen zur Fehlerprotokollierung. Die statische Methode `getLogger` der `Logger`-Klasse liefert ein `Logger`-Objekt. Untersuchen Sie die Eigenschaften der Klasse für die textbasierte Protokollierung zur Fehlerdiagnose. Welche Bedeutung haben die unterschiedlichen Protokollebenen? Wie unterscheidet sich die Methode `info` von der Methode `warning`? Kann die Protokollierung ab- und wieder angeschaltet werden?

14.3.2 Den Klienten informieren

Ein komplett anderer Ansatz als die bisher diskutierten ist, wenn der Dienstleister einen aufgetretenen Fehler an seinen Klienten meldet. Es gibt zwei Wege, wie dies erfolgen kann:

- Ein Dienstleister kann den Ergebniswert einer Methode ungleich `void` benutzen, um entweder den Erfolg oder den Misserfolg des Methodenaufrufs zu signalisieren.
- Ein Dienstleister kann in der Methode *eine Exception werfen*, wenn etwas schiefgeht. Das ist ein für uns neues Java-Konzept, das es auch in anderen Programmiersprachen gibt. Wir werden es ausführlich in Abschnitt 14.4 erläutern.

Beide Techniken haben den Vorteil, dass sie den Programmierer des *Klienten* dazu auffordern, sich auch über das mögliche Scheitern eines Aufrufs Gedanken zu machen. Allerdings hält nur das Werfen einer Exception einen Klienten aktiv davon ab, einen fehlerhaften Aufruf zu ignorieren.

Der erste Ansatz ist leicht umzusetzen, wenn die betrachtete Methode einen Ergebnistyp `void` hat, wie etwa `entferneKontakt`. Wenn das `void` durch den Typ `boolean` ersetzt wird, dann kann die Methode `true` liefern, sofern das Entfernen erfolgreich war, und `false` würde signalisieren, dass etwa schiefgelaufen ist (Listing 14.3).

Listing 14.3

Ein boolescher Ergebnistyp signalisiert Erfolg oder Misserfolg.

```
/**
 * Entferne den Eintrag mit dem gegebenen Schlüssel aus
 * diesem Adressbuch. Bei einem unbekannten Schlüssel
 * tue nichts.
 * @param schlüssel einer der Schlüssel des Eintrags,
 *                  der entfernt werden soll
 * @return true falls der Kontakt erfolgreich entfernt wurde,
 *         sonst false
 */
public boolean entferneKontakt(String schluesssel)
{
    if(schluessselBekannt(schluesssel)) {
        Kontakt kontakt = buch.get(schluesssel);
        buch.remove(kontakt.gibName());
        buch.remove(kontakt.gibTelefon());
        anzahlEintraege--;
        return true;
    }
    else {
        return false;
    }
}
```

Dies ermöglicht einem Klienten, mit einer `if`-Anweisung weitere Anweisungen zu schützen, die vom erfolgreichen Entfernen eines Eintrags abhängen:

```

if(adressbuch.entferneKontakt("...")) {
    // Eintrag erfolgreich entfernt; normal weiterarbeiten.
    ...
}
else {
    // Entfernen schlug fehl; sinnvolle Fehlerbehandlung, falls möglich.
    ...
}

```

Wenn eine Methode des Dienstleisters bereits einen Ergebnistyp ungleich `void` hat – und damit einen booleschen Wert zur Fehlermeldung verhindert – kann immer noch auf andere Weise über den Ergebnistyp signalisiert werden, dass etwas schiefgegangen ist. Das ist dann der Fall, wenn der Wertebereich des Ergebnistyps einen Wert für fehlerhafte Situationen anbietet. Beispielsweise liefert die Methode **gibKontakt** ein **Kontakt**-Objekt für den gegebenen Schlüssel zurück; der folgende Quelltextabschnitt beispielsweise geht davon aus, dass ein spezifischer Schlüssel einen gültigen Kontakt zurückliefert:

```

// Sende eine Nachricht an Andreas
Kontakt kontakt = adressbuch.gibKontakt("Andreas");
String telefon = kontakt.gibTelefon();
...

```

Eine Möglichkeit für die Methode **gibKontakt**, einen ungültigen Schlüssel zu signalisieren, besteht darin, statt eines **Kontakt**-Objekts den Wert `null` zurückzuliefern (Listing 14.4).

```

/**
 * Schlage einen Namen oder eine Telefonnummer
 * nach und liefere den zugehörigen Kontakt.
 * @param schluessel der Name oder die Nummer zum Nachschlagen
 * @return den zum Schlüssel gehörenden Kontakt oder null,
 *         falls der Schlüssel nicht bekannt ist
 */
public Kontakt gibKontakt(String schluessel)
{
    if (schluesselBekannt(schluessel)) {
        return buch.get(schluessel);
    }
    else {
        return null;
    }
}

```

Listing 14.4

Signalisieren eines Fehlers durch ein `null`-Ergebnis.

Dies würde es einem Klienten ermöglichen, in Abhängigkeit vom Aufrufergebnis entweder den normalen Kontrollfluss fortzusetzen oder in geeigneter Weise auf den Fehler zu reagieren:

```

Kontakt kontakt = adressbuch.gibKontakt("Andreas");
if(kontakt != null) {
    // Sende eine Nachricht an Andreas
    String telefon = kontakt.gibTelefon();
    ...
}
else {
    // Eintrag nicht gefunden; sinnvolle Fehlerbehandlung, falls möglich.
    ...
}

```

Es ist durchaus üblich, dass Methoden, die eine Objektreferenz zurückliefern, den Wert `null` zum Signalisieren einer Fehlersituation verwenden. Bei Methoden, die Werte der primitiven Typen zurückliefern, gibt es oft einen Wert, der eine Grenzüberschreitung repräsentiert: Beispielsweise liefert die Methode `indexOf` der Klasse `String` einen negativen Wert als ein Signal, dass das gesuchte Zeichen nicht gefunden wurde.

Übung 14.19 Denken Sie, dass die unterschiedlichen Oberflächentypen der V2T- und V2G-Projekte auch zu unterschiedlichen Arten von Fehlermeldungen führen sollten?

Übung 14.20 Ändern Sie die Klasse `Adressbuch` in einer Kopie des Projekts `Adressbuch-V2T` so ab, dass einem Klienten an geeigneter Stelle mitgeteilt wird, dass eine Methode ungültige Parameterwerte bekommen hat oder aus einem anderen Grund ihren Dienst nicht erfüllen kann.

Übung 14.21 Denken Sie, dass ein Aufruf der Methode `suche`, der keine passenden Kontakte findet, zu einer Fehlermeldung führen sollte? Rechtfertigen Sie Ihre Antwort.

Übung 14.22 Gibt es Kombinationen von Parameterwerten, die Ihrer Meinung nach ungeeignet sind, um bei einem Konstruktoraufruf der Klasse `Kontakt` benutzt zu werden?

Übung 14.23 Verfügt ein Konstruktor über Möglichkeiten, einem Klienten zu signalisieren, dass er mit den gegebenen Parameterwerten den Objektzustand nicht sinnvoll initialisieren kann? Wie sollte ein Konstruktor reagieren, wenn er ungültige Parameterwerte übergeben bekommt?

Offensichtlich kann dieser Ansatz (Anzeige eines Fehlers durch einen Wert, der außerhalb des gültigen Wertebereichs liegt) nicht verwendet werden, wenn alle Werte des Ergebnistyps bereits eine gültige Bedeutung für den Klienten haben. In solchen Fällen bleibt keine andere Wahl als das *Auslösen einer Exception* (siehe Abschnitt 14.4), das auch einige weitere Vorteile bietet. Damit diese Vorteile deutlich werden, heben wir hier zwei Aspekte der Technik mit dem Ergebniswert hervor:

- Es gibt keine Möglichkeit, einen Klienten explizit zum Überprüfen des Ergebniswerts aufzufordern. Deshalb kann ein Klient weiterarbeiten, als wäre nichts geschehen; dies könnte in einer `NullPointerException` enden oder – noch schlimmer – der Fehlerwert könnte als ein regulärer Ergebniswert verwendet werden und zu schwer zu entdeckenden logischen Fehlern führen!
- In einigen Fällen könnte der Fehlerwert für zwei sehr unterschiedliche Zwecke verwendet werden. Das gilt beispielsweise für die überarbeiteten Methoden `gibKontakt` (Listing 14.3) und `entferneKontakt` (Listing 14.4). Einerseits soll dem Klienten signalisiert werden, ob die Anfrage erfolgreich war oder nicht. Andererseits soll signalisiert werden, dass die Anfrage fehlerhaft war, etwa aufgrund ungültiger Parameterwerte.

In den meisten Fällen stellt eine *nicht erfolgreiche* Anfrage keinen logischen Programmierfehler dar, eine *fehlerhafte* Anfrage hingegen meist schon. Wir sollten in diesen beiden Fällen von einem Klienten zwei sehr unterschiedliche Reaktionen erwarten. Es gibt keine generelle Möglichkeit, dies befriedigend über Ergebniswerte zu behandeln.

14.4 Prinzipien der Exception-Behandlung

Das Werfen einer Exception ist die effektivste Möglichkeit für einen Dienstleister, einem Klienten zu signalisieren, dass seine Anfrage nicht bearbeitet werden konnte. Einer der wichtigsten Vorteile gegenüber speziellen Ergebniswerten ist, dass es für einen Klienten (fast) unmöglich ist, eine aufgetretene Exception zu ignorieren und unberührt weiterzuarbeiten. Wenn der Klient die Exception nicht behandelt, dann wird die laufende Anwendung automatisch und unmittelbar beendet.¹ Außerdem kann der Exception-Mechanismus unabhängig von bereits bestehenden Ergebnistypen verwendet werden.

Ein wichtiger Punkt, den Sie während der folgenden Diskussion berücksichtigen sollten, ist, dass im Zusammenhang mit Exceptions der Ort, an dem ein Fehler festgestellt wird, sich von dem Ort unterscheidet, an dem versucht wird, den Fehler zu beheben (sofern dies vorgesehen ist). Die Feststellung eines Fehlers erfolgt in einer Methode des Dienstleisters und die Fehlerbehebung obliegt dem Klienten. Wenn es möglich wäre, den Fehler gleich dort zu beheben, wo er festgestellt wurde, gäbe es keinen Grund, eine Exception auszulösen.

14.4.1 Das Auslösen einer Exception

Listing 14.5 zeigt, wie eine Exception mit einer *throw-Anweisung* geworfen wird. In der Methode `gibKontakt` wird im Falle eines `null`-Parameters eine Exception geworfen, um zu signalisieren, dass die Übergabe von `null` für den Schlüssel eine ungültige Anfrage ist.

```
/**
 * Schlage einen Namen oder eine Telefonnummer
 * nach und liefere den zugehörigen Kontakt.
 * @param schluessel der Name oder die Nummer zum Nachschlagen
 * @return den zum Schlüssel gehörenden Kontakt oder null,
 *         falls der Schlüssel nicht bekannt ist
 * @throws IllegalArgumentException wenn schluessel null ist.
 */
public Kontakt gibKontakt(String schluessel)
{
    if (schluessel == null) {
        throw new IllegalArgumentException(
            "Parameter in gibKontakt ist null.");
    }
    return buch.get(schluessel);
}
```

Listing 14.5

Eine Exception wird geworfen.

¹ Dies ist genau das, was Ihnen immer dann passiert ist, wenn Ihr Programm aufgrund einer `NullPointerException` oder `IndexOutOfBoundsException` ungewollt abgestürzt ist.

Konzept

Eine **Exception** ist ein Objekt, das Informationen über einen Programmfehler hält. Eine Exception wird ausgelöst, um zu signalisieren, dass ein Fehler aufgetreten ist.

Das Auslösen einer Exception besteht aus zwei Schritten: Zuerst wird mit dem Schlüsselwort **new** ein Exception-Objekt erzeugt (in diesem Fall ein Objekt der Klasse **IllegalArgumentException**), und dann wird dieses Exception-Objekt mit dem Schlüsselwort **throw** geworfen. Diese beiden Schritte werden fast immer in einer einzigen Anweisung vorgenommen:

```
throw new ExceptionType("optionaler Fehlertext für die Diagnose");
```

Bei der Erzeugung eines Exception-Objekts kann ein Fehlertext an den Konstruktor übergeben werden. Dieser String kann später vom Empfänger der Exception wieder über die Methoden **getMessage** oder **toString** abgefragt werden. Der String wird dem Benutzer auch dann angezeigt, wenn die Exception nicht behandelt wird und zur Beendigung des Programms führt. Der hier von uns verwendete Exception-Typ **IllegalArgumentException** ist in dem Paket **java.lang** definiert und wird oft dazu verwendet, um anzuzeigen, dass einer Methode oder einem Konstruktor ein nicht korrekter Parameterwert übergeben wurde.

Listing 14.5 zeigt außerdem, dass in der Dokumentation einer Methode zusätzlich über das **javadoc**-Schlüsselwort **@throws** angegeben werden kann, welche Exceptions diese Methode im Falle eines Fehlers wirft.

14.4.2 Geprüfte und ungeprüfte Exceptions

Eine Exception ist immer eine Instanz einer Klasse aus einer speziellen Vererbungshierarchie. Wir können neue Exception-Typen definieren, indem wir Subklassen dieser Hierarchie erzeugen (Abbildung 14.1). Genau genommen ist jede Exception-Klasse eine Subklasse der Klasse **Throwable**, die im Paket **java.lang** definiert ist.² Üblicherweise folgen wir der Konvention, dass neue Exception-Klassen als Subklassen der Klasse **Exception** deklariert werden, die ebenfalls in **java.lang** definiert ist. Das Paket **java.lang** definiert eine Reihe von häufig auftretenden Exception-Klassen, die Ihnen während der Programmierung möglicherweise schon begegnet sind, etwa **NullPointerException**, **IndexOutOfBoundsException** und **ClassCastException**.

Java unterteilt die Exception-Klasse in zwei Kategorien: *geprüfte Exceptions* (*checked exception*) und *ungeprüfte Exceptions* (*unchecked exception*). Alle Subklassen der Klasse **RuntimeException** definieren ungeprüfte Exceptions; alle anderen Subklassen von **Exception** definieren geprüfte Exceptions.

Der Unterschied, etwas vereinfacht dargestellt, ist der folgende: Geprüfte Exceptions sind für solche Fälle gedacht, in denen ein Klient damit rechnen sollte, dass eine Operation fehlschlagen könnte (wenn wir beispielsweise auf eine Festplatte schreiben, dann wissen wir, dass die Platte voll sein könnte). In diesen Fällen sollte ein Klient dazu gezwungen werden, den Erfolg der Operation zu überprüfen. Ungeprüfte Exceptions sind gedacht für Fälle, die im normalen Betrieb eigentlich nicht auftreten sollten – sie werden meist durch Programmfehler ausgelöst. So würde beispielsweise ein Programmierer niemals mit Absicht versuchen, auf ein Element an einer nicht vorhandenen Position in der Liste zuzugreifen. Passiert es dennoch, löst dies folglich eine ungeprüfte Exception aus.

² **Exception** ist eine der beiden direkten Subklassen von **Throwable**; die andere ist **Error**. Subklassen von **Error** sind üblicherweise für Fehler des Laufzeitsystems vorgesehen und nicht für Fehler, auf die ein Programmierer Einfluss hat.

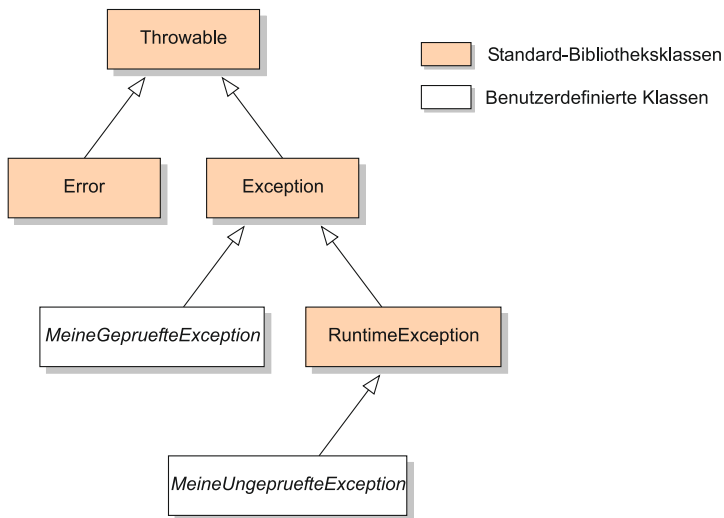


Abbildung 14.1
Die Hierarchie der
Exception-Klassen.

Leider gibt es keine klar definierten Regeln, aus welcher Kategorie im jeweiligen Fall eine Exception ausgelöst werden soll, aber wir können einige allgemeine Hinweise geben:

- Eine Faustregel besagt, dass ungeprüfte Exceptions in Situationen verwendet werden sollten, die zu einem Programmabbruch führen könnten – typischerweise, weil angenommen werden muss, dass ein logischer Fehler im Programm vorliegt, der eine weitere Ausführung unmöglich macht. Daraus folgt, dass geprüfte Exceptions immer dann verwendet werden sollten, wenn ein Problem aufgetreten ist, das ein Klient sinnvoll behandeln kann. Die Schwierigkeit mit dieser Richtlinie ist, dass sie davon ausgeht, dass ein Dienstleister genug über den Kontext weiß, in dem er benutzt wird, und aufgrund dieses Wissens einschätzen kann, ob ein Wiederaufsetzen nach einer Exception möglich ist oder nicht.
- Eine zweite Faustregel besagt, dass ungeprüfte Exceptions in Situationen benutzt werden sollten, die vermeidbar gewesen wären. Beispielsweise ist der Aufruf einer Methode an einer Variablen, die `null` enthält, ein logischer Programmierfehler, der vollständig vermeidbar ist; dass für diesen Fehlerfall die ungeprüfte Exception `NullPointerException` verwendet wird, passt dann zu dieser Regel. Daraus folgt, dass geprüfte Exceptions für Fälle verwendet werden sollten, die außerhalb des Einflussbereichs des Programmierers liegen, etwa eine volle Platte bei einem Schreibzugriff auf das Dateisystem oder ein fehlgeschlagener Netzwerkzugriff aufgrund einer Verbindungsunterbrechung.

Die formalen Regeln im Umgang mit Exceptions unterscheiden sich sehr stark für geprüfte und ungeprüfte Exceptions, und wir werden diese Unterschiede ausführlich in den Abschnitten 14.4.4 und 14.5.1 herausarbeiten. Hier nur kurz zusammengefasst: Der Klient einer Methode, die eine geprüfte Exception werfen kann, muss zwei Dinge tun: Er muss sowohl Anweisungen programmieren, die eine solche Exception erwarten, als auch Anweisungen, die mit der Exception umgehen, wenn sie tatsächlich auftritt.³

³ Leider ist es nach wie vor noch viel zu einfach, dass der Programmierer eines Klienten sich formal an diese Regeln hält, ohne jedoch wirklich eine angemessene Fehlerbehandlung durchzuführen.

Übung 14.24 Nennen Sie drei Exception-Typen aus dem Paket `java.io`.

Übung 14.25 Ist `SecurityException` aus dem Paket `java.lang` eine geprüfte oder eine ungeprüfte Exception? Was ist mit `NoSuchMethodException`? Begründen Sie Ihre Antwort.

14.4.3 Die Auswirkungen einer Exception

Was passiert, wenn eine Exception ausgelöst wird? Zwei Effekte müssen hier betrachtet werden: die Auswirkungen auf die Methode, in der das Problem festgestellt wurde, und die Auswirkungen auf den Aufrufer der problematischen Methode.

Wenn eine Exception ausgelöst wird, wird die Ausführung der auslösenden Methode sofort beendet – sie wird nicht bis zum Ende des Methodenrumpfes ausgeführt. Eine Konsequenz daraus ist, dass eine Methode mit einem anderen Ergebnistyp als `void` kein Ergebnis zurückliefern muss, wenn eine Exception ausgelöst wurde. Das ist durchaus sinnvoll, denn das Auslösen einer Exception ist ein Signal dafür, dass eine Methode ihre Aufgabe nicht korrekt ausführen konnte, somit also auch kein Ergebnis liefern kann. Wir können dieses Prinzip verdeutlichen, indem wir eine geänderte Version des Methodenrumpfes aus Listing 14.5 betrachten:

```
if(schluesseel == null) {
    throw new IllegalArgumentException("Parameter in gibKontakt ist null.");
}
else {
    return buch.get(schluesseel);
}
```

Das Fehlen einer `return`-Anweisung in dem Zweig, in dem die Exception geworfen wird, ist korrekt. Tatsächlich würde der Compiler einen Fehler melden, falls weitere Anweisungen nach der `throw`-Anweisung folgten, denn diese Anweisungen könnten niemals erreicht werden.

Die Auswirkungen auf die Stelle im Programm, an der die Methode aufgerufen wurde, sind etwas komplexer. Insbesondere hängen sie davon ab, ob an dieser Stelle Anweisungen geschrieben wurden, die die Exception *fangen*. Betrachten Sie folgenden konstruierten Aufruf von `gibKontakt`:

```
Kontakt kontakt = adressen.gibKontakt(null);
// Die folgenden Anweisungen können nicht erreicht werden.
String telefon = kontakt.gibTelefon();
```

Wir *können* sagen, dass auf jeden Fall die Ausführung dieser Anweisungen nicht vollständig erfolgt – die Exception, die von `gibKontakt` geworfen wird, unterbricht die Ausführung der ersten Anweisung und es erfolgt keine Zuweisung an die Variable `kontakt`. Als Konsequenz wird auch die zweite Anweisung nicht ausgeführt.

Dieses Beispiel zeigt die Mächtigkeit von Exceptions: Sie halten einen Klienten davon ab, unbeirrt fortzufahren, wenn ein Fehler aufgetreten ist. Was als Nächs-

tes passiert, hängt davon ab, ob die Exception gefangen wird oder nicht. Wenn sie nicht gefangen wird, dann wird das Programm schlicht beendet, mit dem Hinweis, dass eine **NullPointerException** aufgetreten ist. Das Fangen einer Exception werden wir in Abschnitt 14.5.2 diskutieren.

14.4.4 Ungeprüfte Exceptions

Ungeprüfte Exceptions sind aus Sicht des Programmiers am einfachsten zu handhaben, weil ein Compiler nur wenige Überprüfungen für sie vornimmt. So soll das Adjektiv „ungeprüft“ auch verstanden werden – der Compiler führt statisch (zur Übersetzungszeit) keine Überprüfungen durch, weder in der Methode, in der die ungeprüfte Exception geworfen wird, noch an der Stelle, an der die Methode aufgerufen wird. Eine Exception-Klasse definiert ungeprüfte Exceptions, wenn sie eine Subklasse der Klasse **RuntimeException** aus dem Paket **java.lang** ist. Alle Beispiele, mit denen wir bisher das Auslösen einer Exception demonstriert haben, haben ungeprüfte Exceptions verwendet. Deshalb gibt es an dieser Stelle wenig über das Auslösen einer ungeprüften Exception zu sagen: Verwenden Sie einfach eine **throw**-Anweisung.

Wenn wir auch der Konvention folgen, dass ungeprüfte Exceptions immer dann verwendet werden sollten, wenn das Ergebnis voraussichtlich ein Programmabbruch ist – die Exception also voraussichtlich nicht gefangen wird –, dann gibt es auch nicht viel darüber zu sagen, was der Aufrufer tun sollte; denn er wird nichts tun und das Programm wird abgebrochen. Wenn allerdings der Bedarf besteht, dass auch eine ungeprüfte Exception gefangen werden soll, dann *kann* eine Exception-Behandlung, genau wie für eine geprüfte Exception, implementiert werden. Wie dies geschieht, werden wir in Abschnitt 14.5.2 sehen.

Die ungeprüfte **IllegalArgumentException** haben wir bereits kennengelernt. Diese wird geworfen, wenn ein Konstruktor oder eine Methode signalisieren will, dass die übergebenen Parameterwerte ungültig sind. Beispielsweise könnte die Methode **gibKontakt** diese Exception werfen, wenn der übergebene Schlüssel eine leere Zeichenkette ist (Listing 14.6).

```
/**
 * Schlage einen Namen oder eine Telefonnummer
 * nach und liefere den zugehörigen Kontakt.
 * @param schluessel der Name oder die Nummer zum Nachschlagen
 * @throws IllegalArgumentException wenn schluessel leer ist
 * @return den zum Schlüssel gehörenden Kontakt oder null,
 *         falls der Schlüssel nicht bekannt ist
 */
public Kontakt gibKontakt(String schluessel)
{
    if (schluessel == null) {
        throw new IllegalArgumentException("Parameter in gibKontakt" +
                                         "ist null.");
    }
    if (schluessel.trim().length() == 0) {
        throw new IllegalArgumentException("Leerer Schlüssel in gibKontakt.");
    }
    return buch.get(schluessel);
}
```

Konzept

Ungeprüfte Exceptions sind eine Art von Exception, bei deren Verwendung der Compiler keine zusätzlichen Überprüfungen vornimmt.

Listing 14.6

Eine Prüfung auf einen ungültigen Parameterwert.

Es ist durchaus angemessen, dass eine Methode eine Reihe von Prüfungen an ihren Parameterwerten vornimmt, bevor sie ihre eigentliche Aufgabe ausführt. Dies macht es unwahrscheinlicher, dass die Methode erst dann eine Exception aufgrund ungültiger Parameter auslösen muss, wenn sie ihre Aufgabe bereits teilweise ausgeführt hat. Solchen Situationen soll insbesondere deshalb vorgebeugt werden, weil eine unvollständige Änderung des Objektzustands das Objekt möglicherweise in einem inkonsistenten Zustand hinterlässt. Wenn eine Operation aus irgendeinem Grund fehlschlägt, dann sollte das Objekt möglichst in dem Zustand belassen werden, in dem es sich vor dem Aufruf befunden hat.

Die statische `requireNonNull`-Methode der Klasse `java.util.Objects` bietet eine praktische Kurzform, um einen Ausdruck zu testen und eine Exception zu werfen, falls der Ausdruck `null` ist. Der folgende Ausdruck wird eine `NullPointerException` zusammen mit der entsprechenden Zeichenkette ausgeben, falls der Wert von `schluessel` `null` ist; andernfalls passiert nichts:

```
Objects.requireNonNull(schluessel, "null-Schluessel an gibDetails uebergeben");
```

Übung 14.26 Überprüfen Sie alle Methoden der Klasse `Adressbuch` darauf, ob einige von ihnen eine `IllegalArgumentException` auslösen sollten. Falls ja, dann fügen Sie die notwendigen Prüfungen und `throw`-Anweisungen ein.

Übung 14.27 Wenn Sie dies noch nicht getan haben, dann fügen Sie auch die entsprechenden `javadoc`-Kommentare ein, die die möglichen Exceptions erläutern.

Übung 14.28 `UnsupportedOperationException` ist eine ungeprüfte Exception, die in dem Paket `java.lang` definiert ist. Wie kann sie in einer Implementierung des Interface `java.util.Iterator` verwendet werden, um zu verhindern, dass Elemente aus einer Sammlung entfernt werden, über die gerade iteriert wird? Versuchen Sie dies in der Klasse `LogdateiLeser` des Projekts *Weblog-Auswertung* in Kapitel 7.

14.4.5 Objekterzeugung verhindern

Eine wichtige Verwendung von Exceptions dient dazu, die Erzeugung von Objekten zu verhindern, die nicht in einen gültigen Anfangszustand versetzt werden können. Üblicherweise ist das der Fall, wenn einem Konstruktor ungültige Parameterwerte übergeben werden. Wir können dies an der Klasse `Kontakt` verdeutlichen. Der Konstruktor ist momentan recht nachsichtig mit seinen Parameterwerten: Er weist `null`-Werte nicht zurück, sondern ersetzt sie durch leere Zeichenketten. Allerdings benötigt das Adressbuch mindestens einen gültigen Namen oder eine gültige Telefonnummer als eindeutigen Schlüssel, da ein Eintrag mit leerem Namen und leerer Telefonnummer nicht indiziert werden kann. Wir können diese Anforderung verdeutlichen, indem wir in einem solchen Fall die Objekterzeugung verweigern. Das Auslösen einer Exception aus einem Konstruktor erfolgt genau wie aus einer Methode heraus. Listing 14.7 zeigt den überarbeiteten Konstruktor der Klasse `Kontakt`, der verhindert, dass ein Kontakt einen leeren Namen und eine leere Telefonnummer haben kann.

```

/**
 * Lege Kontaktdaten an. Bei allen Angaben werden umgebende
 * Leerzeichen entfernt. Entweder der Name oder die Telefonnummer
 * darf nicht leer sein.
 * @param name      der Name
 * @param telefon    die Telefonnummer
 * @param adresse    die Adresse
 * @throws IllegalStateException wenn name und telefon leer sind
 */
public Kontakt(String name, String telefon, String adresse)
{
    // Leere Strings verwenden, wenn einer der Parameter null ist.
    if(name == null) {
        name = "";
    }
    if(telefon == null) {
        telefon = "";
    }
    if(adresse == null) {
        adresse = "";
    }

    this.name = name.trim();
    this.telefon = telefon.trim();
    this.adresse = adresse.trim();

    if(this.name.isEmpty() && this.telefon.isEmpty()) {
        throw new IllegalStateException(
            "name und telefon dürfen nicht beide leer sein.");
    }
}

```

Listing 14.7

Der Konstruktor der Klasse **Kontakt**.

Eine Exception, die von einem Konstruktor geworfen wird, hat die gleiche Auswirkung wie eine Exception aus einer Methode. Deshalb wird der folgende Versuch, ein ungültiges **Kontakt**-Objekt zu erzeugen, fehlschlagen – er wird *nicht* dazu führen, dass in der Variablen ein **null**-Wert gespeichert wird:

```
Kontakt falscherKontakt = new Kontakt("", "", "");
```

14.5 Die Behandlung von Exceptions

Während für das Werfen einer Exception in Java für geprüfte und ungeprüfte Exceptions die gleichen Regeln gelten, fordern die Sprachregeln lediglich für geprüfte Exceptions, dass sie behandelt werden müssen. Eine Klasse für geprüfte Exceptions ist eine Subklasse der Klasse **Exception**, aber nicht Subklasse von **RuntimeException**. Bei der Benutzung von geprüften Exceptions gibt es etliche Regeln zu beachten, da der Compiler Überprüfungen sowohl in der auslösenden Methode als auch beim Aufrufer durchführt.

14.5.1 Geprüfte Exceptions: die throws-Klausel

Die erste Forderung des Compilers ist, dass eine Methode, die eine geprüfte Exception werfen kann, diesen Umstand durch eine *throws-Klausel* im Kopf der Methode deklarieren muss. Beispielsweise würde eine Methode, die die Exception **IOException** aus dem Paket **java.io** auslösen kann, folgenden Kopf besitzen:⁴

```
public void speichereInDatei(String dateiname)
    throws IOException
```

⁴ Beachten Sie, dass hier das Schlüsselwort **throws** statt **throw** verwendet wird.

Konzept

Geprüfte Exceptions sind Exception-Typen, bei deren Verwendung der Compiler zusätzliche Überprüfungen durchführt und einfordert. Insbesondere erfordern geprüfte Exceptions in Java, dass **throws**-Klauseln und **try**-Blöcke angegeben werden müssen.

Es ist zulässig, eine **throws**-Klausel auch für eine ungeprüfte Exception anzugeben, aber der Compiler fordert dies nicht. Wir empfehlen, in einer **throws**-Klausel nur die geprüften Exceptions aufzuführen, die durch die Methode ausgelöst werden können.

Es sollte unbedingt zwischen der **throws**-Klausel im Kopf der Methode und dem **javadoc**-Kommentar unmittelbar vor der Methode unterschieden werden; der Letztere ist für beide Exception-Arten völlig optional. Wir empfehlen dennoch, dass der entsprechende **javadoc**-Kommentar sowohl für geprüfte als auch für ungeprüfte Exceptions angegeben wird. Auf diese Weise werden einem Benutzer dieser Methode so viele Informationen wie möglich zur Verfügung gestellt.

14.5.2 Exceptions fangen: der try-Block

Die zweite Anforderung ist, dass der Aufrufer einer Methode, die eine geprüfte Exception werfen kann, Maßnahmen für den Umgang mit dieser Exception ergreifen muss. Üblicherweise geschieht dies durch einen sogenannten *Exception-Handler* in Form eines *try*-Blocks. Die meisten **try**-Blöcke haben in der Praxis die allgemeine Form aus Listing 14.8. Hier werden zwei neue Schlüsselwörter eingeführt – **try** und **catch** –, die eine *try-Klausel* bzw. eine *catch-Klausel* einleiten. Beide Klauseln bestehen aus einem Block von Anweisungen.

Listing 14.8

Die **try**- und die **catch**-Klausel eines Exception-Handlers.

```
try {
    //eine oder mehrere geschützte Anweisungen
}
catch(ExceptionTyp e) {
    //die Exception melden und eventuell wieder aufsetzen
}
```

Konzept

Ein Programmabschnitt, der Anweisungen schützt, in denen eine Exception ausgelöst werden kann, wird **Exception-Handler** genannt. Er definiert Anweisungen zur Meldung und/oder zum Wiederaufsetzen nach einer aufgetretenen Exception.

Angenommen, wir haben eine Methode, die den Inhalt eines Adressbuches in einer Datei speichert. Der Benutzer wird in irgendeiner Form nach dem Namen einer Datei gefragt (vielleicht über ein GUI-Dialogfenster) und die Adressbuch-Methode **speichereInDatei** wird dann aufgerufen, um die Liste in die Datei zu schreiben. Wenn wir keine Exceptions zu berücksichtigen hätten, sähe der Quelltext wie folgt aus:

```
dateiname = Dateiname vom Benutzer erfragen
adressbuch.speichereInDatei(dateiname);
```

Weil jedoch der Schreibprozess mit einer geprüften Exception fehlschlagen kann, muss der Aufruf von **speichereInDatei** durch einen **try**-Block umklammert werden, um anzuzeigen, dass dieser Fall berücksichtigt wurde. Listing 14.9 veranschaulicht, wie wir dies schreiben würden, um möglichen Programmfehlern vorzubeugen.

```
String dateiname = null;
boolean erfolgreich;
```

```
try {
    dateiname = // Dateiname vom Benutzer erfragen
    adressbuch.speichereInDatei(dateiname);
    erfolgreich = true;
}
catch(IOException e) {
    System.out.println("Speichern in " + dateiname + " schlug fehl.");
    erfolgreich = false;
}
```

Listing 14.9

Ein Exception-Handler.

Beachten Sie, dass ein **try**-Block eine beliebige Anzahl an Anweisungen enthalten kann, sodass wir dort meistens nicht nur eine Anweisung unterbringen, die fehlschlagen kann, sondern alle Anweisungen, die in irgendeiner Weise damit verbunden sind. Der Grundgedanke ist, dass ein **try**-Block eine Folge von Aktionen repräsentiert, die wir als logische Einheit behandelt möchten, die aber zu irgendeinem Punkt fehlschlagen können.⁵ Der **catch**-Block wird dann versuchen, die Situation zu behandeln oder das Problem zu melden, wenn eine Exception von irgendeiner der Anweisungen im dazugehörigen **try**-Block ausgelöst wird. Beachten Sie, dass die Variablen **dateiname** und **erfolgreich** in diesem Beispiel aus Gründen der Sichtbarkeit außerhalb der **try**-Anweisung deklariert werden müssen, da **try**- und **catch**-Blöcke darauf zugreifen.

Um die Arbeitsweise eines Exception-Handlers zu verstehen, muss man sich klar machen, dass *eine Exception den normalen Kontrollfluss beim Aufrufer unterbricht*. Eine Exception bricht beim Aufrufer die Anweisung ab, die den Aufruf ausführt, und folglich werden alle folgenden Anweisungen nicht mehr ausgeführt. Daraus ergibt sich die Frage: „Wo wird die Ausführung denn fortgesetzt?“ Ein **try**-Block liefert die Antwort: Wenn eine Exception aufgrund eines Aufrufs in einem **try**-Block ausgelöst wird, dann wird die Ausführung in der zugehörigen **catch**-Klausel fortgesetzt. Wenn wir also das Beispiel aus Listing 14.9 betrachten, dann führt das Werfen einer **IOException** durch den Aufruf von **speichereInDatei** dazu, dass der Kontrollfluss aus dem **try**-Block in den **catch**-Block übergeht, wie in Listing 14.10 dargestellt.

```

1. Exception wird hier ausgelöst          2. Ausführung wird hier fortgesetzt
try {
    dateiname = // Dateiname vom Benutzer erfragen
    adressbuch.speichereInDatei(dateiname);
    erfolgreich = true;
}
catch(IOException e) {
    System.out.println("Speichern in " + dateiname + " schlug fehl.");
    erfolgreich = false;
}
  
```

Listing 14.10
Wechsel des
Kontrollflusses in
einem **try**-Block.

Die Anweisungen in einer **try**-Klausel werden auch *geschützte Anweisungen* genannt. Wenn keine Exception während der Ausführung der geschützten Anweisungen auftritt, dann wird die **catch**-Klausel am Ende des **try**-Blocks übergangen. Die Ausführung wird durch den Teil fortgesetzt, der auf den kompletten **try-catch**-Abschnitt folgt.

Eine **catch**-Klausel benennt den Typ der Exception, die durch die Klausel behandelt werden soll, innerhalb von geschweiften Klammern unmittelbar nach dem Schlüsselwort **catch**. Neben dem Exception-Typ wird auch ein Variablenname angegeben (oft einfach nur **e** oder **ex**), über den im **catch**-Block auf die geworfene Exception zugegriffen werden kann. Durch den Zugriff auf die Exception können nützliche Informationen abgefragt werden, die ein Wiederaufsetzen oder die Meldung des Problems ermöglichen – zum Beispiel der Zugriff auf irgendwelche Informationen über den Fehler, die von der werfenden Methode dort platziert wurden. Nachdem

⁵ In **Übung 14.31** finden Sie ein Beispiel für das, was passieren kann, wenn eine Anweisung, die zu einer Exception führen könnte, getrennt von den umliegenden Anweisungen behandelt wird.

die **catch**-Klausel abgearbeitet wurde, kehrt der Kontrollfluss *nicht* wieder an die Stelle zurück, an der die Exception ausgelöst wurde.

Übung 14.29 Im Projekt *Adressbuch-V3T* werden an einigen Stellen ungeprüfte Exceptions geworfen, wenn der Wert eines Parameters **null** ist. Das Projekt enthält auch eine geprüfte Exception-Klasse **KeinPassenderKontaktException**, die nicht benutzt wird. Verändern Sie die Methode **entferneKontakt** der Klasse **Adressbuch** so, dass sie diese Exception auslöst, wenn der übergebene Schlüssel nicht bekannt ist. Fügen Sie einen Exception-Handler in die Methode **entferneEintrag** der Klasse **AdressbuchTexteingabe** ein, der diese Exception fängt und ihr Auftreten meldet.

Übung 14.30 Benutzen Sie die **KeinPassenderKontaktException** in der Methode **aaendereKontakt** von **Adressbuch**. Erweitern Sie die Textschnittstelle so, dass die Daten eines bestehenden Eintrags geändert werden können. Lassen Sie in **AdressbuchTexteingabe** die Exceptions fangen und melden, die durch einen unbekannten Schlüssel ausgelöst werden.

Übung 14.31 Warum ist Folgendes keine sinnvolle Benutzung eines Exception-Handlers? Lässt sich dieser Quelltext übersetzen und ausführen?

```
Person p;
try {
    p = datenbank.finde(personendetails);
}
catch (Exception e) {
}
System.out.println("Die Personendetails gehören zu: " + p);
```

Beachten Sie, dass in allen Beispielen zu den **try**-Anweisungen, die Sie bisher kennengelernt haben, die Exceptions nicht *direkt* von den Anweisungen in der **try**-Klausel geworfen werden. Vielmehr werden die Exceptions *indirekt* ausgelöst – als Reaktion einer Methode, die von den Anweisungen in der **try**-Klausel aufgerufen wurde. Dies entspricht dem normalen Muster und es ist fast immer ein Programmierfehler, eine **throw**-Anweisung direkt in einen **try**-Block zu stellen.

14.5.3 Mehrere Exceptions werfen und fangen

Manchmal wirft eine Methode mehr als einen Exception-Typ, um verschiedene Fehlersituationen zu signalisieren. Wenn es sich dabei um geprüfte Exceptions handelt, dann müssen diese vollständig in der **throws**-Klausel aufgeführt werden, voneinander durch ein Komma getrennt. Beispielsweise:

```
public void verarbeite()
    throws EOFException, FileNotFoundException
```

Ein Exception-Handler muss alle geprüften Exceptions berücksichtigen, die durch die geschützten Anweisungen ausgelöst werden können. Ein **try**-Block kann des-

halb mehrere **catch**-Klauseln enthalten, wie in Listing 14.11 gezeigt. Beachten Sie, dass derselbe Variablenname für die verschiedenen Fälle verwendet werden kann.

```
try {
    ...
    ref.verarbeite();
    ...
}
catch(EOFException e) {
    // angemessene Behandlung einer Dateiende-Exception (end-of-file)
    ...
}
catch(FileNotFoundException e) {
    // angemessene Behandlung für eine nicht gefundene Datei
    ...
}
```

Listing 14.11

Mehrere **catch**-Klauseln in einem **try**-Block.

Wenn eine Exception durch eine der geschützten Anweisungen eines **try**-Blocks ausgelöst wird, werden die **catch**-Klauseln in der Reihenfolge, in der sie im Quelltext stehen, nach einer zu der Exception passenden Klausel durchsucht. So würde in diesem Fall bei einer **EOFException** die erste Klausel ausgewählt werden und bei der **FileNotFoundException** würde der Kontrollfluss bei der zweiten fortgesetzt. Sobald das Ende einer einzelnen **catch**-Klausel erreicht wurde, wird die Ausführung nach der letzten **catch**-Klausel fortgesetzt.

Mit Polymorphie kann bei Bedarf vermieden werden, dass mehrere **catch**-Klauseln angegeben werden müssen. Dies geschieht dann aber möglicherweise auf Kosten der Möglichkeit, typspezifisch reagieren zu können. In Listing 14.12 wird eine einzelne **catch**-Klausel definiert, die *alle* Exceptions der geschützten Anweisungen fängt. Das funktioniert deshalb, weil bei der Suche nach einer passenden **catch**-Klausel lediglich geprüft wird, ob das Exception-Objekt eine Instanz des in der Klausel benannten Typs ist. Da alle Exception-Klassen Subtypen der Klasse **Exception** sind, wird die einzelne Klausel alle denkbaren Exceptions fangen – egal, ob geprüft oder ungeprüft. Aufgrund der beschriebenen Suchstrategie folgt, dass die Reihenfolge der **catch**-Klauseln relevant ist und dass die **catch**-Klausel eines bestimmten Typs nicht auf eine **catch**-Klausel eines seiner Supertypen folgen kann – denn die vorher genannte Supertyp-Klausel wird immer gewählt werden, bevor die Subtyp-Klausel geprüft wird. Der Compiler wird dies als Fehler melden.

```
try {
    ...
    ref.verarbeite();
    ...
}
catch(Exception e) {
    // angemessene Behandlung aller Exceptions
    ...
}
```

Listing 14.12

Fangen aller Exceptions in einer einzigen **catch**-Klausel.

Eine bessere Möglichkeit, mehrere Exceptions mit der gleichen Aktion zum Wiederaufsetzen oder Melden zu verarbeiten, ist, die Exception-Typen zusammen aufzulisten. Die verschiedenen Exception-Typen stehen vor dem Variablennamen der Exception und werden durch das Symbol **|** getrennt (Listing 14.13).

Listing 14.13
Fangen mehrerer
Exceptions.

```
try {
    ...
    ref.verarbeite();
    ...
}
catch(IOException | FileNotFoundException e) {
    // angemessene Behandlung beider Exceptions
    ...
}
```

Übung 14.32 Verändern Sie die **try**-Blöcke Ihrer Lösungen zu den **Übungen 14.29** und **14.30** so, dass geprüfte und ungeprüfte Exceptions in unterschiedlichen **catch**-Klauseln verarbeitet werden.

Übung 14.33 Was ist falsch am folgenden **try**-Block?

```
Person p = null;
try {
    p = datenbank.finde(personendetails);
    System.out.println("Die Personendetails gehören zu: " + p);
}
catch(Exception e) {
    // Behandle jede geprüfte Exception ...
    ...
}
catch(RuntimeException e) {
    // Behandle jede ungeprüfte Exception ...
    ...
}
```

14.5.4 Eine Exception propagieren

Bisher haben wir gesagt, dass eine Exception so früh wie möglich gefangen und behandelt werden sollte. Also sollte eine Exception, die durch die Methode **verarbeite** ausgelöst wurde, in der Methode gefangen und behandelt werden, die **verarbeite** aufgerufen hat. Tatsächlich ist das jedoch keine strikte Sprachregel, denn Java erlaubt, dass eine Methode, die eine Exception auslöst, diese an ihre eigenen Aufrufer *propagieren* kann. Eine Methode propagiert eine Exception einfach durch den Umstand, dass sie keinen Exception-Handler für die Exception implementiert. Für geprüfte Exceptions fordert der Compiler dann jedoch, dass diese Methode in ihrem eigenen Kopf eine **throws**-Klausel angibt, obwohl sie die Exception nicht selbst erzeugt und auslöst. Das bedeutet, dass Sie manchmal auf eine Methode stoßen, die eine **throws**-Klausel aufweist, aber keine **throw**-Anweisung im Methodenrumpf enthält. Ein Propagieren ist üblich, wenn die aufrufende Methode die Exception entweder nicht behandeln kann oder nicht behandeln muss, dies aber weiter oben in der Aufrufkette möglich oder notwendig ist. Es ist auch üblich in Konstruktoren, bei denen das Einrichten eines neuen Objekts fehlschlägt und der Konstruktor anschließend nicht wiederaufsetzen kann.

Wenn es sich um eine ungeprüfte Exception handelt, dann ist die **throws**-Klausel wieder optional; wir lassen sie dann üblicherweise aus.

14.5.5 Die finally-Klausel

Ein **try**-Block kann einen dritten Abschnitt enthalten, der optional ist. Es ist die *finally-Klausel* (Listing 14.14), die häufig ausgelassen wird. Mit einer **finally**-Klausel können Anweisungen angegeben werden, die auf jeden Fall ausgeführt werden sollen, unabhängig davon, ob eine Exception aufgetreten ist oder nicht. Wenn der Kontrollfluss das Ende einer **try**-Klausel erreicht, dann werden die **catch**-Klauseln ignoriert, die **finally**-Klausel wird jedoch ausgeführt. Tritt andererseits eine Exception auf, dann wird die passende **catch**-Klausel ausgewählt und anschließend die **finally**-Klausel ausgeführt.

```
try {
    // eine oder mehrere geschützte Anweisungen
}
catch(Exception e) {
    // die Exception melden und eventuell wieder aufsetzen
}
finally {
    // gemeinsame Aktionen, die ausgeführt werden müssen,
    // unabhängig davon, ob eine Exception aufgetreten ist oder nicht
}
```

Listing 14.14

Ein **try**-Block mit einer **finally**-Klausel.

Auf den ersten Blick erscheint eine **finally**-Klausel redundant. Führt das folgende Beispiel nicht zur gleichen Ausführungsreihenfolge wie die Anweisungen in Listing 14.14?

```
try {
    eine oder mehrere geschützte Anweisungen
}
catch(Exception e) {
    die Exception melden und eventuell wieder aufsetzen
}
gemeinsame Aktionen, die ausgeführt werden müssen,
unabhängig davon, ob eine Exception aufgetreten ist oder nicht
```

Tatsächlich gibt es sogar zwei Situationen, in denen diese beiden Quelltextabschnitte sich unterschiedlich verhalten:

- Eine **finally**-Klausel wird auch dann ausgeführt, wenn eine **return**-Anweisung in der **try**- oder der **catch**-Klausel ausgeführt wird.
- Wenn in der **try**-Klausel eine Exception ausgelöst wird, die nicht gefangen wird, dann wird dennoch die **finally**-Klausel ausgeführt.

Im zweiten Fall könnte die nicht aufgefangene Exception eine ungeprüfte Exception sein, für die keine **catch**-Klausel angegeben werden muss. Es könnte sich aber auch um eine geprüfte Exception handeln, die nicht durch eine der **catch**-Klauseln gefangen, aber von der Methode propagiert wird, um weiter oben in der Aufrufkette behandelt zu werden. Auch in einem solchen Fall würde die **finally**-Klausel noch ausgeführt.

Es kann also **try**-Blöcke ohne **catch**-Klauseln geben, die sowohl eine **try**-Klausel als auch eine **finally**-Klausel haben, wenn die Methode alle Exceptions propagiert:

```
try {
    eine oder mehrere zu schützende Anweisungen
}
```

```

finally {
    Aktionen, die ausgeführt werden müssen,
    unabhängig davon, ob eine Exception aufgetreten ist oder nicht
}

```

14.6 Neue Exception-Klassen definieren

Falls die vordefinierten Exception-Klassen die Ursache eines Problems nicht ausreichend beschreiben, können mithilfe von Vererbung zusätzliche Exception-Klassen definiert werden, die bessere Informationen liefern. Zusätzliche Klassen für geprüfte Exceptions können als Subklassen jeder bereits bestehenden Klasse für geprüfte Exceptions (etwa **Exception**) definiert werden, zusätzliche ungeprüfte Exceptions wären Subklassen in der Hierarchie von **RuntimeException**.

Alle bestehenden Exception-Klassen bieten die Möglichkeit, dem Konstruktor eine Fehlerbeschreibung zu übergeben. Einer der Hauptgründe für das Definieren neuer Exception-Klassen ist jedoch, dem Exception-Objekt zusätzliche Informationen mitzugeben, die die Diagnose und das Wiederaufsetzen erleichtern. Beispielsweise erwartet eine Methode wie **aendereKontakt** in der *Adressbuch*-Anwendung einen Parameter **schluessel**, der auf einen existierenden Eintrag passen sollte. Wenn kein passender Eintrag existiert, ist das ein Programmierfehler, da die Methode ihre Aufgabe nicht erfüllen kann. Beim Melden dieses Fehlers ist es nützlich, wenn der Schlüssel, der den Fehler verursacht hat, mitgeliefert wird. Listing 14.15 zeigt eine neue Klasse für geprüfte Exceptions, die im Projekt *Adressbuch-V3T* definiert wird. Sie bekommt als Konstruktorparameter den Schlüssel und macht diesen sowohl über den Diagnose-String als auch über eine speziell definierte sondierende Methode zugänglich. Wenn diese Exception durch einen Exception-Handler gefangen wird, steht der Schlüssel für die Anweisungen zur Verfügung, die ein Wiederaufsetzen versuchen sollen.

Listing 14.15

Eine Exception-Klasse mit zusätzlichen Diagnose-Informationen.

```

/**
 * Diese Exception hält den Schlüssel, für den keine
 * passenden Kontaktdaten im Adressbuch gefunden werden
 * konnten.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class KeinPassenderKontaktException extends Exception
{
    // Der Schlüssel ohne passende Kontaktdaten.
    private String schluessel;

    /**
     * Speichere Fehlerinformationen.
     * @param schluessel der Schlüssel ohne passenden Kontakt
     */
    public KeinPassenderKontaktException(String schluessel)
    {
        this.schluessel = schluessel;
    }

    /**
     * @return den fehlerhaften Schlüssel
     */
    public String gibSchluessel()
    {
        return schluessel;
    }
}

```

```

/**
 * @return einen Diagnosetext mit dem fehlerhaften Schlüssel
 */
public String toString()
{
    return "Keine passender Kontakt für '"
        + schluesssel + "' gefunden.";
}
}

```

Dieses Prinzip, Informationen zum leichteren Wiederaufsetzen mitzuliefern, sollte insbesondere bei Klassen für geprüfte Exceptions beachtet werden. Durch das Definieren formaler Parameter für den Exception-Konstruktor kann gewährleistet werden, dass solche Diagnoseinformationen auch zur Verfügung stehen. Zusätzlich können durch Überschreiben der Methode `toString` nützliche Informationen für die Fehlerdiagnose geliefert werden, auch wenn kein Wiederaufsetzen möglich oder notwendig ist.

Übung 14.34 Definieren Sie im Projekt *Adressbuch-V3T* eine neue Klasse für geprüfte Exceptions: `DoppelterSchluesselException`. Diese sollte durch die Methode `neuerKontakt` ausgelöst werden, wenn eines der beiden nichtleeren Schlüsselfelder des neuen Kontakts einen bereits bekannten Schlüssel enthält. Die Exception-Klasse sollte die Werte der falschen Schlüssel speichern. Nehmen Sie alle notwendigen Änderungen an der Klasse für die Benutzungsschnittstelle vor, um die Exception fangen und melden zu können.

Übung 14.35 Denken Sie, dass die Exception `DoppelterSchluesselException` eine geprüfte oder eine ungeprüfte Exception sein sollte? Begründen Sie Ihre Antwort.

14.7 Die Verwendung von Zusicherungen

14.7.1 Interne Konsistenzüberprüfungen

Wenn wir eine Klasse entwerfen oder implementieren, dann haben wir häufig ein intuitives Gefühl für Aussagen über den Quelltext, die zu einem bestimmten Zeitpunkt der Ausführung wahr sein sollten; aber wir formulieren diese selten formal. Beispielsweise würden wir bei einem Objekt der Klasse `Kontakt` erwarten, dass mindestens eines seiner Datenfelder nicht leer ist; oder dass nach einem Aufruf der Methode `entferneKontakt` mit einem bestimmten Schlüssel dieser Schlüssel anschließend nicht mehr in Gebrauch ist. Typischerweise wollen wir solche Bedingungen festlegen, während die Klasse entwickelt wird, bevor sie ausgeliefert wird. In gewisser Weise sind die Tests, die wir in Kapitel 9 diskutiert haben, solche Versuche, ein sinnvolles Verhalten einer Klasse oder einer Methode formal festzulegen. Charakteristisch für die dort diskutierten Tests ist, dass sie *außerhalb* der zu testenden Klasse definiert sind. Wenn eine Klasse verändert wurde, dann sollten wir uns Zeit für die Ausführung der Regressionstests nehmen, um uns davon zu überzeugen, dass die Klasse weiterhin korrekt funktioniert; das wird leicht vergessen. Die Art der Parameterüberprüfung, die wir in

diesem Kapitel eingeführt haben, verschiebt die Betonung bereits etwas von rein äußerlichen Überprüfungen hin zu einer Kombination von externer und interner Überprüfung. Allerdings dient Parameterüberprüfung in erster Linie dem Schutz eines Dienstleisters vor fehlerhafter Benutzung durch einen Klienten. Dies lässt die Frage offen, ob wir auch interne Prüfungen einführen können, die das korrekte Verhalten des Dienstleisters überprüfen.

Wir könnten solche internen Prüfungen während der Entwicklung mithilfe der normalen Exception-Mechanismen implementieren. In der Praxis müssten wir dafür ungeprüfte Exceptions verwenden, denn wir können nicht fordern, dass Klienten Exception-Handler für Fehler formulieren müssen, die intern in einem Dienstleister auftreten. Dann müssten wir uns mit der Frage beschäftigen, ob wir diese internen Prüfungen entfernen, sobald der Entwicklungsprozess abgeschlossen wurde, damit wir die möglicherweise hohen Kosten für die Überprüfungen zur Laufzeit vermeiden können.

14.7.2 Die `assert`-Anweisung für Zusicherungen

Für effiziente interne Konsistenzprüfungen, die während der Entwicklung angeschaltet, aber für die Auslieferung eines Programms abgeschaltet werden können, bietet Java einen Zusicherungsmechanismus in Form der *assert-Anweisung* an. Eine ähnliche Idee ist uns bereits in Kapitel 9 im Zusammenhang mit den JUnit-Tests begegnet, in denen wir die Ergebnisse zusicherten, die wir von den Methodenaufrufen erwarteten, und das JUnit-Rahmenwerk testete, ob diese Zusicherungen bestätigt wurden.

Das Projekt *Adressbuch-Assert* ist eine Entwicklungsversion der *Adressbuch*-Projekte, in dem die Verwendung von solchen Zusicherungen demonstriert wird. Listing 14.16 zeigt die Methode `entferneKontakt`, die zwei Formen von *assert-Anweisungen* enthält.

Listing 14.16
assert-Anweisungen
für interne Konsistenz-
prüfungen.

```
/**
 * Entferne den Eintrag mit dem gegebenen Schlüssel aus diesem
 * Adressbuch. Der Schlüssel sollte diesem Adressbuch bekannt sein.
 * @param schlüssel einer der Schlüssel des Eintrags,
 *                der entfernt werden soll
 * @throws IllegalArgumentException wenn schlüssel null ist
 */
public void entferneKontakt(String schlüssel)
{
    if(schlüssel == null){
        throw new IllegalArgumentException(
            "Parameter 'schlüssel' in entferneKontakt ist null.");
    }
    if(schlüsselBekannt(schlüssel)) {
        Kontakt kontakt = buch.get(schlüssel);
        buch.remove(kontakt.gibName());
        buch.remove(kontakt.gibTelefon());
        anzahlEintraege--;
    }
    assert !schlüsselBekannt(schlüssel);
    assert konsistenteGroesse() : "Inkonsistente Buchgröße in entferneKontakt";
}
```

Das Schlüsselwort **assert** wird gefolgt von einem booleschen Ausdruck. Mit einer solchen Anweisung soll ausgedrückt werden, dass an dieser Stelle in der Methode eine bestimmte Bedingung zugesichert wird. Beispielsweise sichert die erste **assert**-

Anweisungen in Listing 14.16 zu, dass `schlüsselBekannt` an dieser Stelle `false` liefern sollte, entweder weil der Schlüssel von vornherein nicht bekannt war oder weil er nach dem Entfernen des Kontakts nicht mehr in Benutzung ist. Diese anscheinend offensichtliche Zusicherung ist wichtiger, als sie auf den ersten Blick erscheinen mag; beachten Sie, dass beim Entfernen selbst der Schlüssel nicht direkt beteiligt ist.

Eine `assert`-Anweisung dient somit zwei Zwecken. Sie drückt etwas aus, das wir an einem bestimmten Punkt der Ausführung als wahr annehmen, und erhöht somit die Lesbarkeit sowohl für den aktuellen Entwickler als auch für zukünftige Wartungsprogrammierer; und sie führt darüber hinaus die Überprüfung der Zusicherung durch, sodass wir benachrichtigt werden, wenn unserer Annahmen nicht zutreffen. Dies kann sehr dabei helfen, Fehler früh und leicht zu finden.

Wenn der boolesche Ausdruck in einer `assert`-Anweisung als wahr ausgewertet wird, dann hat die `assert`-Anweisung keinen weiteren Effekt. Wenn das Ergebnis jedoch `false` ist, dann wird ein `AssertionError` ausgelöst. Die zugehörige Klasse ist eine Subklasse von `Error` (siehe Abbildung 14.1) und somit Teil der Hierarchie, die die nicht behebbaren Fehler repräsentiert: Folglich sollte ein Klient keinen Exception-Handler dafür vorsehen.

Die zweite `assert`-Anweisung Listing 14.16 illustriert die alternative Form einer `assert`-Anweisung. Der String, der auf den Doppelpunkt folgt, wird als Parameter an den Konstruktor von `AssertionError` übergeben, um einen Diagnosetext zu liefern. Der zweite Ausdruck muss kein expliziter String sein; jeder Ausdruck mit einem Ergebnis ist zulässig und wird in einen `String` umgewandelt, der an den Konstruktor übergeben wird. Die erste `assert`-Anweisung zeigt, dass eine Zusicherung häufig eine existierende Methode innerhalb der Klasse benutzt (`schlüsselBekannt`). Das zweite Beispiel illustriert, dass es nützlich sein kann, spezifisch für die Überprüfung einer Zusicherung eine eigene Methode anzubieten (`konsistenteGroesse` in diesem Beispiel). Dies kann sinnvoll sein, wenn die Überprüfung umfangreiche Berechnungen vornimmt. Listing 14.17 zeigt die Methode `konsistenteGroesse`, die überprüfen soll, ob das Datenfeld `anzahlEintraege` korrekt die Anzahl der eindeutigen Kontakte im Adressbuch wiedergibt.

Konzept

Eine **Zusicherung** ist eine Tatsachenaussage, die bei normaler Programmausführung zutreffen sollte. Wir können Zusicherungen verwenden, um unsere Annahmen explizit zu formulieren und um Programmierfehler leichter entdecken zu können.

```
/**
 * Überprüfe, ob das Datenfeld anzahlEintraege konsistent
 * ist mit der Anzahl der momentan gespeicherten Einträge
 * im Adressbuch.
 * @return true, falls das Datenfeld konsistent ist, false sonst
 */
private boolean konsistenteGroesse()
{
    Collection<Kontakt> alleEintraege = buch.values();
    // Entfernen von Duplikaten, da wir mehrere Schlüssel verwenden.
    Set<Kontakt> ohneDuplikate = new HashSet<Kontakt>(alleEintraege);
    int momentaneEintraege = ohneDuplikate.size();
    return anzahlEintraege == momentaneEintraege;
}
```

Listing 14.17

Interne Konsistenzprüfung im Adressbuch.

14.7.3 Richtlinien zur Benutzung von Zusicherungen

Zusicherungen dienen primär der Überprüfung von Konsistenzbedingungen während der Entwicklungs- und Testphase eines Projektes. Sie sind nicht für ausgelieferten Code gedacht. Aus diesem Grund fügt ein Java-Compiler die `assert`-Anweisungen nur auf explizite Anweisung hin in den übersetzten Code ein. Daraus folgt, dass

assert-Anweisungen niemals dazu verwendet werden sollten, reguläre Funktionalität zur Verfügung zu stellen. Beispielsweise wäre es falsch, Zusicherungen mit dem Entfernen eines Kontaktes aus einem Adressbuch zu verknüpfen, wie in folgendem Beispiel:

```
// Fehler: assert niemals für normale Ausführung verwenden!
assert buch.entferne(Kontakt.gibName()) != null;
assert buch.entferne(Kontakt.gibTelefon()) != null;
```

Übung 14.36 Öffnen Sie das Projekt *Adressbuch-Assert*. Sehen Sie sich die Klasse **Adressbuch** genau an und machen Sie sich bei allen **assert**-Anweisungen klar, was und warum geprüft wird.

Übung 14.37 Die Klasse **AdressbuchDemo** enthält mehrere Testmethoden, die Methoden von **Adressbuch** mit **assert**-Anweisungen darin aufrufen. Sehen Sie sich den Quelltext der Klasse **AdressbuchDemo** an und machen Sie sich die Tests klar und führen Sie anschließend jede Testmethode aus. Werden Zusicherungsfehler generiert? Wenn ja, verstehen Sie, weshalb?

Übung 14.38 Die Methode **aendereKontakt** in **Adressbuch** enthält momentan keine **assert**-Anweisung. Eine Zusicherung, die wir vornehmen könnten, ist, dass das Adressbuch am Ende dieselbe Anzahl an Einträgen haben sollte wie zu Anfang der Methode. Fügen Sie eine **assert**-Anweisung (und weitere Anweisungen, die notwendig sind) ein, die dies überprüft. Führen Sie anschließend die Methode **testAendern** aus **AdressbuchDemo** aus. Denken Sie, dass diese Methode auch eine Überprüfung auf konsistente Größe enthalten sollte?

Übung 14.39 Angenommen, wir entscheiden, dass ein Adressbuch nicht nur über Name und Telefonnummer, sondern auch über die Adresse indizierbar sein sollte. Wenn wir einfach die Anweisung

```
buch.put(kontakt.gibAdresse(), kontakt);
```

in der Methode **neuerKontakt** einfügen, vermuten Sie, dass eine der Zusicherungen fehlschlägt? Probieren Sie es aus! Nehmen Sie alle notwendigen Änderungen an **Adressbuch** vor, die dazu führen, dass alle Zusicherungen zutreffen.

Übung 14.40 Objekte der Klasse **Kontakt** sind unveränderlich – sie haben keine verändernden Methoden. Wie wichtig ist dieser Umstand für die interne Konsistenz der Klasse **Adressbuch**? Nehmen Sie beispielsweise an, die Klasse würde eine Methode **setzeTelefon** anbieten – können Sie Tests formulieren, die die Probleme dieses Umstandes aufdecken könnten?

14.7.4 Zusicherungen und JUnit-Tests in BlueJ

In Kapitel 9 haben wir die Unterstützung vorgestellt, die BlueJ für das Testrahmenwerk JUnit anbietet. Diese Unterstützung basiert auf dem Zusicherungsmechanismus, den wir in diesem Abschnitt diskutiert haben. Methoden von JUnit wie **assert**-

Equals sind um **assert**-Anweisungen herum formuliert, deren boolescher Ausdruck aus den jeweiligen Parametern der Methoden zusammengesetzt ist. Wenn Testklassen von JUnit zum Testen von Klassen benutzt werden, die eigene **assert**-Anweisungen enthalten, dann werden Zusicherungsfehler aus diesen Anweisungen im Fenster für Testergebnisse neben den Fehlern aus Zusicherungen in der Testklasse angezeigt. Das Projekt *Adressbuch-JUnit* enthält eine Testklasse, die diese Kombination demonstriert. Die Methode **testNeuerKontaktError** aus **AdressbuchTest** wird einen **AssertionError** auslösen, weil die Methode **neuerKontakt** nicht verwendet werden sollte, um einen existierenden Eintrag zu ändern (siehe **Übung 14.34**).

14.8 Wiederaufsetzen und Fehlervermeidung

Bisher haben wir uns in diesem Kapitel darauf konzentriert, dass der Dienstleister Fehler erkennt und dem Klienten diese Fehler zurückmeldet. Zwei weitere wichtige Begriffe beim Umgang mit Fehlern sind das Wiederaufsetzen und die Fehlervermeidung.

14.8.1 Wiederaufsetzen

Die erste Voraussetzung für ein erfolgreiches Wiederaufsetzen nach einem Fehler ist, dass der Klient den Fehler überhaupt bemerkt. Das mag offensichtlich klingen, aber es kommt durchaus häufig vor, dass ein Klient sich auf die ordnungsgemäße Ausführung einer gerufenen Methode verlässt und den Ergebniswert gar nicht überprüft. Obwohl Exceptions sehr viel schwieriger zu ignorieren sind, haben wir schon mehrfach das folgende Muster für Exception-Handler zu sehen bekommen:

```
Kontakt kontakt = null;
try {
    kontakt = adressbuch.gibKontakt(...);
}
catch(Exception e) {
    System.out.println("Fehler: " + e);
}
String telefon = kontakt.gibTelefon();
```

Obwohl die Exception gefangen und gemeldet wird, wird nicht berücksichtigt, dass es eventuell falsch sein kann, einfach weiterzuarbeiten.

Der **try**-Block von Java bietet die Möglichkeit, nach dem Auftreten einer Exception einen Mechanismus zum Wiederaufsetzen zu installieren. Ein Wiederaufsetzen nach einem Fehler umfasst üblicherweise einige korrigierende Anweisungen in dem **catch**-Block und dann einen erneuten Versuch. Wiederholte Versuche können vorgenommen werden, indem der **try**-Block in einer Schleife ausgeführt wird. Ein Beispiel dieses Ansatzes zeigt Listing 14.18, das eine Erweiterung der Version aus Listing 14.9 ist. Die Versuche, alternative Dateinamen zu erstellen, könnten beispielsweise eine Liste möglicher Ordner einbeziehen oder einen interaktiven Benutzer nach einem anderen Namen fragen.

Listing 14.18

Wiederaufsetzen
explizit programmiert.

```
// Versuche, das Adressbuch zu speichern.
boolean erfolgreich = false;
int versuche = 0;
do {
    try {
        adressbuch.speichereInDatei(dateiname);
        erfolgreich = true;
    }
    catch(IOException e) {
        System.out.println("Speichern in " + dateiname + " schlug fehl.");
        versuche++;
        if(versuche < MAX_VERSUCHE) {
            dateiname = // ein alternativer Dateiname
        }
    }
} while(!erfolgreich && versuche < MAX_VERSUCHE);
if(!erfolgreich) {
    //das Problem melden und aufgeben
}
```

Obwohl dieses Beispiel das Wiederaufsetzen in einer sehr spezifischen Situation zeigt, lassen sich doch einige Aspekte verallgemeinern:

- Das Vorhersehen eines Fehlers und der Versuch, nach einem Fehler wieder aufzusetzen, erfordert üblicherweise komplexere Kontrollstrukturen als für Fälle, in denen kein Fehler auftreten kann.
- Die Anweisungen in der **catch**-Klausel stellen die Voraussetzungen für ein Wiederaufsetzen her.
- Wiederaufsetzen bedeutet häufig Wiederholung.
- Erfolgreiches Wiederaufsetzen kann nicht garantiert werden.
- Es sollte verhindert werden, dass ein erfolgloses Wiederaufsetzen in einer endlosen Schleife wiederholt wird.

Es steht nicht immer ein interaktiver Benutzer für alternative Eingaben zur Verfügung, daher kann es in der Verantwortung des Klienten liegen, den Fehler zum Beispiel mittels einer Logging-API zu protokollieren, sodass er bemerkt wird und später untersucht werden kann.

14.8.2 Fehlervermeidung

Es sollte klar geworden sein, dass eine Situation, in der eine Exception aufgetreten ist, schlimmstenfalls zum Programmabbruch führt und bestenfalls nur sehr umständlich wieder bereinigt werden kann. Es kann sehr viel einfacher sein, den Fehler von vornherein zu vermeiden; dies erfordert jedoch die Zusammenarbeit von Klient und Dienstleister.

Viele der Fälle, in denen ein **Adressbuch**-Objekt gezwungen ist, eine Exception auszulösen, beschäftigen sich mit dem Übergeben von **null**-Werten an seine Methoden. Diese Fälle stellen logische Programmierfehler dar, die offensichtlich vermieden werden könnten, wenn der Klient die Parameter vor einem Aufruf des Dienstleisters selbst überprüfen würde. Parameter mit **null**-Werten belegen üblicherweise falsche Annahmen im Klienten. Betrachten Sie beispielsweise folgende Anweisungen:

```
String schluesel = datenbank.suche(postleitzahl);
Kontakt universitaet = buch.gibKontakt(schluesel);
```

Wenn die Suche in der Datenbank fehlschlägt, dann kann der gelieferte Schlüssel sehr leicht entweder leer oder `null` sein. Das direkte Weitergeben dieses Werts an die Methode `gibKontakt` würde zu einer Exception führen. Mit einer einfachen Prüfung des Suchergebnisses kann die Exception jedoch vermieden werden und stattdessen das eigentliche Problem der erfolglosen Suche untersucht werden:

```
String schluesssel = datenbank.suche(postleitzahl);
if(schluesssel != null && !schluesssel.isEmpty() {
    Kontakt universitaet = buch.gibKontakt(schluesssel);
    ...
}
else {
    Umgang mit dem Fehler bei der Postleitzahl ...
}
```

In diesem Fall könnte der Klient selbst herausfinden, dass eine Methode nicht aufgerufen werden sollte. Manchmal ist dies jedoch nicht möglich und manchmal muss der Klient dafür die Hilfe des Dienstleisters in Anspruch nehmen.

In **Übung 14.34** wurde gefordert, dass die Methode `neuerKontakt` keinen Kontakt akzeptieren sollte, wenn einer der beiden Schlüssel bereits im Adressbuch bekannt ist. Um einen solchen ungeeigneten Aufruf zu vermeiden, könnte der Klient die Methode `schluessselBekannt` des Adressbuches folgendermaßen benutzen:

```
// Einen eigentlich neuen Kontakt ins Adressbuch eintragen
if(buch.schluessselBekannt(kontakt.gibName()) {
    buch.aendereKontakt(kontakt.gibName(), kontakt);
}
else if(buch.schluessselBekannt(kontakt.gibTelefon()) {
    buch.aendereKontakt(kontakt.gibTelefon(), kontakt);
}
else {
    den Kontakt neu einfügen ...
}
```

Durch dieses Vorgehen kann offensichtlich vermieden werden, dass die Methode `neuerKontakt` die `DoppelterSchluessselException` wirft, und die Exception kann deshalb von einer geprüften zu einer ungeprüften umgewandelt werden.

Dieses Beispiel verdeutlicht einige allgemeine Prinzipien:

- Wenn die Methoden des Dienstleisters zur Gültigkeits- oder Zustandsüberprüfung auch Klienten zur Verfügung stehen, kann dieser häufig vermeiden, dass Exceptions überhaupt auftreten.
- Wenn eine Exception auf diese Weise vermieden werden kann, dann weist ihr Auslösen auf einen logischen Programmierfehler im Klienten hin. Deshalb kann für solche Fälle eine ungeprüfte Exception verwendet werden.
- Durch Verwendung einer ungeprüften Exception muss der Klient keinen `try`-Block angeben, wenn er für sich bereits sichergestellt hat, dass diese nicht auftreten kann. Das ist ein großer Gewinn, denn für einen Programmierer ist es sehr ärgerlich, wenn er einen `try`-Block programmieren muss, obwohl die entsprechende Exception aus seiner Sicht nicht auftreten kann; das kann dazu führen, dass die Fehlerbehandlung für andere Fehler nicht mehr angemessen ernst genommen wird.

Dieses Vorgehen hat aber nicht nur Vorteile. Hier sind einige Gründe, warum der Ansatz nicht immer praktikabel ist:

- Wenn Methoden des Dienstleisters zur Gültigkeits- oder Zustandsüberprüfung auch Klienten zur Verfügung stehen, kann dies ein Aufbrechen der Kapselung bedeuten und zu einer engeren Kopplung zwischen Klient und Dienstleister führen als erwünscht.
- Ein Dienstleister kann sich möglicherweise nicht darauf verlassen, dass der Klient die Prüfungen zur Vermeidung der Exception wirklich durchführt. Dies führt dann häufig zu einer doppelten Prüfung sowohl im Klienten als auch im Dienstleister. Wenn solche Prüfungen recht „teuer“ (rechenintensiv in ihrer Ausführung) sind, dann sind solche Verdopplungen eventuell unerwünscht oder sogar nicht vertretbar. Aus unserer Sicht gilt: Wenn die Möglichkeit zur Wahl besteht, dann sollte eher ein Teil der Effizienz geopfert werden, wenn dadurch die resultierenden Programme sicherer werden.

14.9 Dateibasierte Ein- und Ausgabe

Ein wichtiger Bereich der Programmierung, in dem die Behandlung von Exceptions nicht ignoriert werden kann, ist die Ein- und Ausgabe. In diesem Bereich hat ein Programmierer meist nur wenig Einfluss auf die Umgebung, in der die Anwendung läuft. Beispielsweise könnte eine Datei, die von einer Anwendung benötigt wird, vor der Ausführung aus Versehen gelöscht oder beschädigt worden sein. Oder der Versuch, eine Datei anzulegen, schlägt aufgrund von fehlenden Zugriffsrechten oder Quota-Regelungen für das Dateisystem fehl. Es gibt viele mögliche Gründe, warum eine Eingabe- oder Ausgabe-Operation fehlschlagen kann. Außerdem ist moderne Ein- und Ausgabe nicht mehr darauf beschränkt, dass ein Programm einfach auf seinen lokalen Dateispeicher zugreift, sondern hat sich auf eine Netzwerkumgebung verlagert, in der die Konnektivität für den Ressourcenzugriff anfällig und inkonsistent ist – wie zum Beispiel im Falle einer mobilen Umgebung.

Die API von Java hat über die Jahre viele Veränderungen erfahren, die die zunehmende Diversität der Umgebungen, in denen Java-Programme inzwischen ausgeführt werden, widerspiegeln. Das wichtigste Paket für ein- und ausgabespezifische Klassen ist seit jeher **java.io**. Dieses Paket enthält eine Reihe von Klassen zur Unterstützung von plattformunabhängigen Eingabe-/Ausgabe-Operationen. In diesem Paket wird auch die geprüfte **IOException** definiert, die als allgemeines Signal verwendet wird, dass eine Ein-/Ausgabe-Operation fehlgeschlagen ist. Fast jede Ein- oder Ausgabe-Operation muss davon ausgehen, dass eine der Exceptions geworfen werden kann. Instanzen von geprüften Subklassen von **IOException** (wie etwa **EOFException** oder **FileNotFoundException**) können gelegentlich geworfen werden, um ausführlichere Diagnoseinformationen zu bieten. In neueren Java-Versionen gab es hinsichtlich der Anzahl an Klassen eine Verschiebung vom Paket **java.io** zur **java.nio**-Hierarchie, ohne jedoch alles vollständig in **java.io** zu ersetzen.

Eine komplette Darstellung der zahlreichen Klassen in den Paketen **java.io** und **java.nio** würde den Rahmen dieses Buches sprengen. Wir beschränken uns an dieser Stelle daher darauf, anhand von bereits bekannten Projekten einige grundlegende Beispiele aufzuzeigen. Sie sollten Ihnen eine ausreichende Grundlage liefern,

um in Ihren eigenen Projekten mit Ein- und Ausgabe experimentieren zu können. Im Einzelnen werden wir folgende typische Dateioperationen demonstrieren:

- Informationen über eine Datei aus dem Dateisystem ziehen
- Text mithilfe der Klasse **FileWriter** in eine Datei schreiben
- Text mithilfe der Klassen **FileReader** und **BufferedReader** aus einer Datei einlesen
- das Vorhersehen von **IOExceptions**
- Eingaben mit der **Scanner**-Klasse parsen

Außerdem wollen wir uns damit beschäftigen, wie Objekte binär in Dateien geschrieben oder binär aus Dateien eingelesen werden können, und uns unter diesem Aspekt die Java-Unterstützung für *Serialisierung* einmal näher ansehen.

Zum weiteren Einlesen in die Eingabe-/Ausgabe-Unterstützung von Java empfehlen wir das Tutorial von Oracle, das zu finden ist unter:

<http://download.oracle.com/javase/tutorial/essential/io/index.html>.

14.9.1 Reader, Writer und Streams

Viele der Klassen im Paket **java.io** fallen in eine von zwei Hauptkategorien: Klassen für den Umgang mit Textdateien und Klassen für den Umgang mit Binärdateien. Wir können uns Textdateien so vorstellen, dass sie Daten im Format des Java-Typs **char** enthalten – typischerweise einfache, zeilenbasierte, menschenlesbare, alphanumerische Informationen. Webseiten in HTML sind ein besonderes Beispiel. Binärdateien sind etwas vielfältiger: Bilddateien sind ein typisches Beispiel oder ausführbare Programme wie ein Textverarbeitungsprogramm oder ein Media-Player. Klassen für den Umgang mit Textdateien werden in Java *Reader* und *Writer* genannt, während Klassen für Binärdateien mit sogenannten *Streams* (im Sinne von Datenströmen) umgehen.⁶ Hier werden wir ausschließlich Reader und Writer betrachten.

14.9.2 Die Klasse File und das Interface Path

Eine Datei ist mehr als nur ein Name und irgendein Inhalt. Eine Datei wird zum Beispiel in einem bestimmten *Ordner* oder *Verzeichnis* auf einem bestimmten Laufwerk gespeichert; außerdem haben verschiedene Betriebssysteme unterschiedliche Konventionen hinsichtlich der Zeichen, die in den Dateipfadnamen erlaubt sind. Die Klasse **File** von **java.io** ermöglicht es einem Programm, sich über eine externe Datei zu informieren, und zwar unabhängig von dem Dateisystem, auf dem das Programm ausgeführt wird. Durch die Erzeugung eines **File**-Objekts in einem Programm wird keine Datei im Dateisystem angelegt. Vielmehr werden Informationen über eine existierende Datei in dem **File**-Objekt abgespeichert, vorausgesetzt die Datei existiert.

Das modernere Interface **Path** sowie die Klasse **Files** (man beachte den Plural) aus **java.nio.file** erfüllen eine ähnliche Rolle für die alte **File**-Klasse. Da **Path** jedoch ein Interface ist und keine Klasse, wird eine konkrete Instanz einer implementierenden Klasse mithilfe einer statischen **get**-Methode der Klasse **Paths** (auch hier Plural) erzeugt, die sich ebenfalls in dem Paket **java.nio.file** befindet.

⁶ Es ist wichtig zu beachten, dass die **Stream**-Klasse zur Ein-/Ausgabe im Paket **java.io** sich von den **Stream**-Klassen im Paket **java.util.stream** unterscheidet.

Wer mit altem Code arbeitet, kann mit der **toPath**-Methode von **File** ein äquivalentes **Path**-Objekt aus einem **File**-Objekt erzeugen.

Wir können manchmal Situationen vermeiden, die eine Exception-Behandlung erfordern, indem wir mit der **Files**-Klasse und einem **Path**-Objekt prüfen, ob eine Datei existiert. Die **Files**-Klasse verfügt über die Methoden **exists** und **isReadable**, die – sofern zuerst verwendet – das Öffnen einer Datei seltener fehlschlagen lassen. Da jedoch das Öffnen einer existierenden Datei *immer noch* voraussetzen kann, dass eine Exception zu behandeln ist, machen sich viele Programmierer nicht die Mühe, vorher zu prüfen.

Die Klasse **Files** bietet eine große Zahl von statischen Methoden, um die Attribute eines **Path**-Objekts abzufragen.

Übung 14.41 Lesen Sie die API-Dokumentation der Klasse **Files** im Paket **java.nio.file**. Welche Art von Information ist verfügbar über Objekte der Klasse **Files/Path**?

Übung 14.42 Mit welchen Methoden der **Files**-Klasse können Sie feststellen, ob ein **Path**-Objekt eine normale Datei oder ein Verzeichnis (Ordner) angibt?

Übung 14.43 Ist es möglich, irgendwelche Informationen über *den Inhalt* einer Datei mithilfe der **Files**-Klasse zu erhalten?

14.9.3 Textausgaben

Zum Speichern von Daten in eine Datei sind drei Schritte nötig:

- die Datei öffnen
- die Daten schreiben
- die Datei schließen

Es liegt in der Natur von Dateiausgaben, dass jeder dieser Schritte fehlschlagen kann, und zwar aus den unterschiedlichsten Gründen, die meist komplett außerhalb des Einflussbereichs des Anwendungsentwicklers liegen. Deshalb muss bei jedem Schritt damit gerechnet werden, dass eine Exception ausgelöst wird.

Um in eine Textdatei zu schreiben, wird üblicherweise ein Objekt der Klasse **FileWriter** erzeugt, das als Konstruktorparameter den Namen der Zieldatei bekommt. Der Dateiname kann entweder in Form eines **String**- oder eines **File**-Objekts angegeben werden. Die Erzeugung eines **FileWriters** führt dazu, dass die externe Datei geöffnet und auf den Empfang von Ausgabedaten vorbereitet wird. Wenn das Öffnen aus irgendeinem Grund fehlschlägt, löst der Konstruktor eine **IOException** aus. Gründe für ein Fehlschlagen können mangelnde Rechte im Dateisystem oder ein ungültiger Pfad der Datei sein.

Wenn eine Datei erfolgreich geöffnet wurde, dann kann die Methode **write** eines **FileWriters** für das Speichern von Zeichen – meist in Form von Strings – in die Datei benutzt werden. Jeder Schreibversuch kann fehlschlagen, auch wenn die Datei erfolgreich geöffnet wurde. Solche Fehlversuche sind selten, aber möglich.

Nachdem alle Daten geschrieben wurden, ist es wichtig, die Datei zu schließen. Dies garantiert, dass die Daten auch wirklich in das externe Dateisystem geschrieben

wurden, und führt üblicherweise auch zur Freigabe von internen und externen Ressourcen. Auch das Schließen der Datei kann in sehr seltenen Fällen fehlschlagen.

Das Grundmuster für den beschriebenen Ablauf sieht also etwa so aus:

```
FileWriter writer = new FileWriter(Dateiname);
while(noch Text zum Ausgeben vorhanden) {
    ...
    writer.write(nächsten Textabschnitt);
    ...
}
writer.close();
```

Das Hauptproblem besteht darin, die während dieser drei Schritte möglicherweise auftretenden Exceptions angemessen behandeln zu können. Eine Exception, die durch das Öffnen der Datei ausgelöst wird, ist die Einzige, die eventuell sinnvoll behandelt werden kann, und das auch nur, wenn ein alternativer Dateiname gewählt werden kann. Da dies üblicherweise erfordert, dass der Benutzer der Anwendung eingreift, hängen die Chancen für eine solche Behandlung sehr stark von der Anwendung und vom jeweiligen Kontext ab. Wenn das Schreiben in die Datei fehlschlägt, dann ist es unwahrscheinlich, dass ein erneuter Versuch erfolgreich sein wird; das Gleiche gilt für das Schließen der Datei. Die Folge ist in der Regel eine unvollständige Datei.

Ein praktisches Beispiel hierfür finden Sie in Listing 14.19. Die Klasse **Logdatei-Erzeuger** im Projekt *Weblog-Auswertung* umfasst eine Methode, um eine Reihe von zufälligen Logeinträgen in eine Datei zu schreiben, deren Name als Parameter der Methode **erzeugeDatei** übergeben wird. Die Methode verwendet zwei verschiedene **write**-Methoden: eine zum Schreiben eines Strings und eine zum Schreiben eines Zeichens. Nachdem wir den Text des Eintrags als String ausgegeben haben, schreiben wir das Zeichen für den Zeilenumbruch in die Datei, damit jeder Eintrag in einer separaten Zeile steht.

```
/**
 * Erzeuge eine Datei mit zufälligen Logeinträgen.
 * @param dateiname die Datei, in die geschrieben werden soll
 * @param anzahlEintraege wie viele Einträge
 * @return true, wenn erfolgreich, ansonsten false
 */
public boolean erzeugeDatei(String dateiname, int anzahlEintraege)
{
    boolean erfolg = false;
    try {
        FileWriter writer = new FileWriter(dateiname);
        Logeintrag[] eintraege = new Logeintrag[anzahlEintraege];
        for(int i = 0; i < anzahlEintraege; i++) {
            writer.write(eintraege[i].toString());
            writer.write('\n');
        }

        writer.close();
        erfolg = true;
    }
    catch(IOException e) {
        System.err.println("Es gab ein Problem beim Schreiben in " +
            dateiname);
    }
    return erfolg;
}
```

Listing 14.19
In eine Textdatei
schreiben.

14.9.4 Die try-Anweisung mit Ressourcen

Es ist wichtig, eine Datei nach Beendigung wieder zu schließen, und es gibt eine Version der **try**-Anweisung, die die Aufgabe vereinfacht, dies sicherzustellen. Diese Art der Anweisung wird **try**-Anweisung mit Ressourcen (*try-with-resources*) oder automatisches Ressourcen-Management (ARM, *Automatic Resource Management*) genannt. Listing 14.20 zeigt, wie sie in der Klasse **LogdateiErzeuger** eingesetzt wird, die Sie im Projekt *Weblog-Auswertung* finden.

Listing 14.20

In eine Textdatei schreiben mithilfe einer **try**-Anweisung mit Ressourcen.

```
/**
 * Erzeuge eine Datei mit zufälligen Logeinträgen.
 * @param dateiname die Datei, in die geschrieben werden soll
 * @param anzahlEintraege wie viele Einträge
 * @return true, wenn erfolgreich, ansonsten false
 */
public boolean erzeugeDatei(String dateiname, int anzahlEintraege)
{
    boolean erfolg = false;
    try (FileWriter writer = new FileWriter(dateiname)) {
        // Erzeugung des Eintrags ausgelassen

        for(int i = 0; i < anzahlEintraege; i++) {
            writer.write(eintraege[i].toString());
            writer.write('\n');
        }
        erfolg = true;
    }
    catch(IOException e) {
        System.err.println("Es gab ein Problem beim Schreiben in " +
            dateiname);
    }
    return erfolg;
}
```

Das Ressourcen-Objekt wird in einer neuen Klammer direkt im Anschluss an das Schlüsselwort **try** erzeugt. In diesem Fall wird die Ressource einer Datei zugeordnet. Sobald die **try**-Anweisung beendet ist, wird auf die Ressource automatisch die Methode **close** aufgerufen. Diese Version der **try**-Anweisung eignet sich nur für Objekte von Klassen, die das Interface **AutoCloseable** aus dem Paket **java.lang** implementieren. Das werden vornehmlich die Klassen sein, die mit Ein- und Ausgabe zu tun haben.

Übung 14.44 Überarbeiten Sie das *Zuul*-Projekt so, dass es die Benutzereingaben protokolliert und in eine Textdatei schreibt (als Aufzeichnungsskript des Spiels). Zur Lösung dieser Aufgabe sind mehrere Lösungsansätze denkbar:

- Sie könnten die **Parser**-Klasse so ändern, dass jede Eingabezeile in einer Liste gespeichert wird, die dann am Ende des Spiels ausgegeben wird. Diese Lösung kommt dem oben angesprochenen einfachen Programmiermuster am nächsten.
- Sie könnten die gesamte Dateibehandlung in die **Parser**-Klasse integrieren, sodass die gelesenen Zeilen ohne weitere Bearbeitung direkt in die Datei geschrieben werden. Das Öffnen, Schreiben und Schließen der Datei werden zeitlich voneinander getrennt (oder wäre es sinnvoll, die Datei für jede zu schreibende Zeile zu öffnen, zu schreiben und zu schließen?)

- Sie könnten die Behandlung der Datei in der **Spiel**-Klasse unterbringen und in der **Befehl**-Klasse eine **toString**-Methode implementieren, die den für jeden Befehl zu schreibenden **String** zurückgibt.

Bewerten Sie jede dieser möglichen Lösungen im Hinblick auf den Entwurf nach Zuständigkeiten und die Exception-Behandlung und überlegen Sie, welche Auswirkungen es für das Spiel hätte, wenn das Skript nicht geschrieben werden könnte. Wie können Sie sicherstellen, dass die Skriptdatei immer am Ende des Spiels geschlossen wird? Das ist wichtig, damit die Ausgabe auch physikalisch in die Datei geschrieben wird.

14.9.5 Einlesen von Text

Das Gegenstück zur Textausgabe in eine Datei mit einem **FileWriter** ist das Einlesen mit einem **FileReader**. Wie leicht zu erraten ist, sind auch hier drei Schritte notwendig: die Datei öffnen, die Daten einlesen, die Datei wieder schließen. Während die natürlichen Einheiten zur Textausgabe Zeichen und Strings sind, sind es beim Einlesen Zeichen und Textzeilen. Aber während die Klasse **FileReader** eine Methode zum Einlesen eines einzelnen Zeichens anbietet,⁷ bietet sie keine Methode zum Lesen einer Zeile. Das Problem beim Einlesen einer Zeile besteht darin, dass es keine Obergrenze für ihre Länge gibt. Somit müsste jede Methode, die die nächste komplette Zeile aus einer Datei liefern soll, eine beliebige Anzahl an Zeichen einlesen können. Aus diesem Grund arbeiten wir üblicherweise mit einem **BufferedReader** (also einen Reader mit einem expliziten Puffer), der eine Methode **readLine** anbietet. Allerdings kann **BufferedReader** keine Dateien öffnen! Deshalb müssen wir zuerst ein **FileReader** erzeugen und es direkt in einem **BufferedReader**-Objekt kapseln.

Ein **BufferedReader** wird mit der Methode **newBufferedReader** der Klasse **Files** erzeugt. Zusätzlich zu einem **Path**-Parameter, der die zu öffnende Datei angibt, kommt erschwerend hinzu, dass ein **Charset**-Parameter erforderlich ist. Es wird benötigt, um den Zeichensatz für die Zeichen in der Datei zu beschreiben. **Charset** ist im Paket **java.nio.charset** zu finden. Es gibt mehrere Standardzeichensätze, wie **US-ASCII** und **ISO-8859-1**, und weitere Informationen hierzu sind in der API-Dokumentation zu **Charset** zu finden. Das Zeichen für das Zeilenende ist im Ergebnis der Methode **readLine** nicht enthalten und ein Ergebniswert **null** signalisiert das Ende der Datei.

Dies legt folgendes Programmiermuster für das Einlesen einer Datei nahe:

```
Charset zeichensatz = Charset.forName("ISO-8859-1");
Path pfd = Paths.get(Dateiname);
BufferedReader reader =
    Files.newBufferedReader(
        pfd, zeichensatz);
```

⁷ Tatsächlich liefert die Methode **read** jedes Zeichen als einen **int**-Wert und nicht als **char**, weil -1 als ein spezieller Wert zur Grenzüberschreitung das Ende der Datei signalisiert. Dies ist genau die Art von Technik, die wir in Kapitel 14.3.2 beschrieben haben.

```

String zeile = reader.readLine();
while(zeile != null) {
    tue etwas mit der Zeile
    zeile = reader.readLine();
}
reader.close();

```

Listing 14.21 demonstriert, wie **BufferedReader** in dem Projekt *Technischer-Kundendienst* aus Kapitel 6 dazu genutzt werden kann, die Standardantworten des Systems aus einer Datei auszulesen, statt sie im Quelltext der **Beantworter**-Klasse hart festlegen zu müssen (Projekt *Technischer-Kundendienst-IO* im Ordner dieses Kapitels).

Listing 14.21

Aus einer
Textdatei mittles
BufferedReader lesen.

```

import java.io.*;
import java.nio.charset.Charset;
import java.nio.file.*;
import java.util.*;

public class Beantworter
{
    // Liste von Standardantworten, falls kein Wort erkannt wird.
    private List<String> standardantworten;

    // Der Name der Datei mit den Standardantworten.
    private static final String DATEI_MIT_STANDARDANTWORTEN = "standardantworten.txt";

    // ... andere Datenfelder und Methoden hier ausgelassen ...

    /**
     * Generiere eine Liste von Standardantworten, aus denen wir eine
     * auswählen können, wenn wir keine bessere Antwort wissen.
     */
    private void standardantwortlisteFuellen()
    {
        Charset Zeichensatz = Charset.forName("ISO-8859-1");
        Path pfad = Paths.get(DATEI_MIT_STANDARDANTWORTEN);
        try(BufferedReader reader = Files.newBufferedReader(pfad,
                                                         Zeichensatz)){
            String antwort = reader.readLine();
            while(antwort != null) {
                standardantworten.add(antwort);
                antwort = reader.readLine();
            }
        }
        catch(FileNotFoundException e) {
            System.err.println(DATEI_MIT_STANDARDANTWORTEN
                               + " kann nicht geöffnet werden");
        }
        catch(IOException e) {
            System.err.println("Beim Lesen von " +
                               DATEI_MIT_STANDARDANTWORTEN +
                               " ist ein Fehler aufgetreten");
        }
        // Sicherstellen, dass mindestens eine Antwort gegeben wird.
        if(standardantworten.size() == 0) {
            standardantworten.add("Könnten Sie sich näher dazu äußern?");
        }
    }
}

```

Wie schon bei der Dateiausgabe stellt sich die Frage, wie mit auftretenden Exceptions umgegangen werden soll. In diesem Beispiel haben wir eine Fehlermeldung ausgegeben und dafür gesorgt, dass bei einem kompletten Fehlschlagen des Lesevorgangs zumindest eine Antwort bereitgestellt wird.

Es gibt sogar einen noch einfacheren Weg, eine vollständige Textdatei einzulesen, ohne **BufferedReader** benutzen zu müssen. Die Methode **lines** der **Files**-Klasse bekommt einen **Path**-Parameter und gibt den Inhalt der Datei in der Form

`Stream<String>` zurück. Dies kann dann als Stream verarbeitet oder in ein `String`-Array umgewandelt oder in einer `ArrayList` gespeichert werden.

Übung 14.45 Die Datei *standardantworten.txt* im Projektordner des Projekts *Technischer-Kundendienst-IO* enthält die Standardantworten, die von der Methode `standardantwortlisteFuellen` eingelesen werden. Ändern Sie mit einem beliebigen Texteditor den Inhalt dieser Datei so, dass sie eine leere Zeile zwischen zwei Antworten enthält. Ändern Sie anschließend Ihren Quelltext, damit er immer noch die Antworten korrekt aus dieser Datei liest.

Übung 14.46 Ändern Sie Ihren Quelltext so, dass mehrere Textzeilen, die in der Datei nicht durch eine Leerzeile getrennt sind, als eine Antwort gelesen werden. Ändern Sie die Datei *standardantworten.txt* so, dass es mehrere Antworten enthält, die sich über mehrere Zeilen erstrecken.

Übung 14.47 Modifizieren Sie die Klasse `Beantworter` des Projekts *Technischer-Kundendienst-IO* so, dass sie die Kombinationen von Schlüsselwörtern und Antworten aus einer Textdatei liest, anstatt `antwortemap` mit Strings zu initialisieren, die im Quelltext der Methode `antwortemapBefuellen` fest vorgegeben sind. Sie können die Methode `standardantwortenlisteFuellen` als Muster nehmen, müssen allerdings einige Änderungen an der Logik vornehmen, da für jeden Eintrag zwei Strings statt einem gelesen werden müssen: das Schlüsselwort und die Antwort. Versuchen Sie, die Schlüsselwörter und die Antworten in abwechselnden Zeilen zu speichern, wobei Sie den Text zu jeder Antwort in einer Zeile halten sollten. Sie können voraussetzen, dass immer eine gerade Anzahl Zeilen vorhanden ist (d.h. es gibt keine fehlenden Antworten).

14.9.6 Scanner: Eingaben parsen

Bisher haben wir Eingaben größtenteils als unstrukturierte Textzeilen behandelt, für die sich `BufferedReader` am besten eignet. Viele Anwendungen gehen jedoch noch einen Schritt weiter und zerlegen die einzelnen Zeilen in mehrere Komponenten, die Datenwerte verschiedener Typen repräsentieren. Das CSV-Format (*comma-separated values*) wird häufig verwendet, um Textdateien zu speichern, deren Zeilen aus mehreren Werten bestehen, die jeweils vom Nachbarwert durch ein Komma getrennt werden.⁸ Dadurch haben solche Textzeilen eine implizite Struktur. Diese zugrunde liegende Struktur zu identifizieren, wird als *Parsen* bezeichnet – im Gegensatz zum *Scannen*, bei dem die einzelnen Zeichen zu getrennten Datenwerten zusammengestellt werden.

Die Klasse `Scanner` im Paket `java.util` wurde speziell dazu entworfen, Text zu scannen und zusammengesetzte Zeichenfolgen in Werte eines Typs zu konvertieren, zum Beispiel ganze Zahlen (`nextInt`) und Fließkommazahlen (`nextDouble`). Während ein `Scanner` dazu verwendet werden kann, um `String`-Objekte zu zerlegen, wird er auch sehr oft dazu genutzt, um den Inhalt von Dateien direkt zu lesen und umzuwandeln, anstatt ein `BufferedReader` einzusetzen. Die dazugehörigen Konstruktoren können

⁸ In der Praxis kann jedes beliebige Zeichen anstelle eines Kommas als Trennzeichen verwendet werden, häufig wird beispielsweise auch ein Tabulatorzeichen verwendet.

String-, **File**- oder auch **Path**-Argumente übernehmen. Listing 14.22 veranschaulicht, wie eine Textdatei, deren Inhalt als ganze Zahlen interpretiert werden soll, auf diese Weise gelesen werden könnte.

Listing 14.22
Ganze Zahlen mit
Scanner lesen.

```
/**
 * Lies ganze Zahlen aus einer Datei und liefere sie als Array zurück.
 * @param dateiname die zu lesende Datei
 * @return die eingelesenen ganzen Zahlen
 */
private int[] liesGanzeZahlen(String dateiname)
{
    int[] daten;
    try {
        List<Integer> werte = new ArrayList<>();
        Scanner scanner = new Scanner(new File(dateiname));
        while(scanner.hasNextInt()) {
            werte.add(scanner.nextInt());
        }
        // in Array passender Größe kopieren
        daten = new int[werte.size()];
        Iterator<Integer> it = werte.iterator();
        int i = 0;
        while(it.hasNext()) {
            daten[i] = it.next();
            i++;
        }
    }
    catch(IOException e) {
        System.out.println("Kann folgende Datei nicht finden: " +
            dateiname);
        daten = new int[0];
    }
    return daten;
}
```

Beachten Sie, dass diese Methode nicht garantiert, dass die Datei vollständig gelesen wird. Die **Scanner**-Methode **hasNextInt**, die die Schleife steuert, liefert **false** zurück, wenn sie in der Datei auf Text trifft, der nicht Teil einer ganzen Zahl zu sein scheint. Damit wird das Sammeln der Daten abgebrochen. Tatsächlich ist es möglich und gar nicht unüblich, beim Parsen einer Datei Aufrufe von verschiedenen **next**-Dateien zu mischen, wenn die darin enthaltenen Daten aus gemischten Typen bestehen.

Eine weitere häufige Verwendung von **Scanner** ist es, Eingaben von der „Konsole“ einzulesen, die mit einem Programm verbunden ist. Wir haben regelmäßig die Methoden **print** und **println** von **System.out** verwendet, um Text auf der BlueJ-Konsole auszugeben. **System.out** ist vom Typ **java.io.PrintStream** und leitet zu dem um, was als Ziel häufig *Standardausgabe* (*standard output*) genannt wird. In ähnlicher Weise existiert eine korrespondierende *Standardeingabe* (*standard input*) als **System.in**, die vom Typ **java.io.InputStream** ist. Ein **InputStream** wird üblicherweise nicht direkt verwendet, wenn Eingaben vom Benutzer gelesen werden sollen, weil er die Eingaben nur zeichenweise liefert. Stattdessen wird **System.in** üblicherweise an den Konstruktor der Klasse **Scanner** übergeben. Die Klasse **Eingabeleser** im Projekt *Technischer-Kundendienst-komplett* aus Kapitel 6 verwendet diesen Ansatz, um Fragen des Benutzers einzulesen:

```
Scanner leser = new Scanner(System.in);
... Code hier ausgelassen ...
String eingabeZeile = leser.nextLine();
```

Die Methode `nextLine` der Klasse `Scanner` liefert die komplette nächste Eingabezeile von der Standardeingabe (ohne das abschließende Zeichen für den Zeilenumbruch).

Übung 14.48 Sehen Sie sich die Klasse `Eingabeleser` aus *Technischer-Kundendienst-komplett* noch einmal genau an und überprüfen Sie, ob Sie die dortige Verwendung der Klasse `Scanner` verstehen.

Übung 14.49 Lesen Sie die Dokumentation der Klasse `Scanner` im Paket `java.util`. Welche `next`-Methoden außer den bisher in diesem Abschnitt erwähnten bietet sie an?

Übung 14.50 Sehen Sie sich in der Klasse `Parser` im Projekt *Zuul-besser* noch einmal an, wie sie ebenfalls einen `Scanner` verwendet. Sie tut dies auf eine leicht andere Weise.

Übung 14.51 Sehen Sie sich die Klasse `LogdateiZeilenerleger` von *Weblog-Auswertung* noch einmal an, wie sie einen `Scanner` verwendet, um ganze Zahlen aus den Logdateizeilen zu extrahieren.

14.9.7 Objektserialisierung

Vereinfacht gesagt, erlaubt Serialisierung das Schreiben ganzer Objekte in eine externe Datei mit einer einzelnen Schreiboperation und das Wiedereinlesen zu einem späteren Zeitpunkt mit nur einer einzigen Leseoperation.⁹ Dies ist sowohl mit einfachen als auch mit zusammengesetzten Objekten wie etwa Sammlungen möglich. Durch diesen mächtigen Mechanismus ist es beispielsweise nicht notwendig, Objekte einzeln Datenfeld für Datenfeld in eine Datei zu schreiben. Der Mechanismus ist für Anwendungen mit persistenten Daten wie Adressbücher oder Media-Datenbanken besonders nützlich, da alle Einträge, die während einer Sitzung erstellt wurden, gespeichert und in der nächsten Sitzung wiederverwendet werden können. Natürlich könnten wir die Inhalte der Objekte auch als Textstrings schreiben, aber das Lesen der Texte, das Umwandeln der eingelesenen Informationen in die richtigen Typen und das Wiederherstellen *des exakten Zustands* eines komplexen Objektsatzes ist oft schwierig, wenn nicht sogar unmöglich. Die Objektserialisierung ist im Vergleich dazu viel zuverlässiger.

Damit eine Klasse bei der Serialisierung berücksichtigt werden kann, muss sie das Interface `Serializable` aus dem Paket `java.io` implementieren. Dabei ist interessant, dass dieses Interface keine Methoden definiert. Der Prozess der Serialisierung wird automatisch durch das Laufzeitsystem kontrolliert und erfordert nur sehr wenig selbst programmierten Code. In unserem *Adressbuch-IO*-Projekt implementieren sowohl `Adressbuch` als auch `Kontakt` dieses Interface, sodass ihre Instanzen per Serialisierung in einer Datei gespeichert werden können. Um zu veranschaulichen, wie eine Serialisierung implementiert wird, definiert die Klasse `Adressbuch-Dateiverwalter` die Methoden `speichereInDatei` und `liesAusDatei`. Listing 14.23

Konzept

Serialisierung

erlaubt, ganze Objekte und Objekthierarchien in einer einzigen Operation zu lesen und zu schreiben. Jedes beteiligte Objekt muss dazu Instanz einer Klasse sein, die das Interface `Serializable` implementiert.

⁹ Dies ist deshalb eine Vereinfachung, weil Objekte beispielsweise auch über eine Netzverbindung geschrieben und gelesen werden können, nicht nur innerhalb eines externen Dateisystems.

enthält den Quelltext von `speichereInDatei` und zeigt, mit wie wenig Code sich das ganze Adressbuch in einer einzigen `write`-Anweisung speichern lässt. Beachten Sie auch, dass statt eines `Writer`-Objekts ein `Stream`-Objekt verwendet wird, weil wir die Objekte in binärer Form schreiben. Die Klasse `AdressbuchDateiverwalter` enthält noch weitere Beispiele für grundlegende Lese- und Schreibtechniken im Zusammenhang mit Textdateien, z.B. die Methoden `speichereSuchergebnisse` und `zeigeSuchergebnisse`.

Listing 14.23

Serialisierung eines kompletten `Adressbuch`-Objekts mit allen Kontaktdaten.

```
public class AdressbuchDateiverwalter
{
    // Datenfelder und Methoden hier ausgelassen

    /**
     * Speichere eine Binärversion des Adressbuchs in
     * der Datei mit dem gegebenen Namen. Wenn der Dateiname
     * nicht absolut ist, wird er als relativ zum aktuellen
     * Projektordner angenommen.
     * @param dateiname      der Name der Datei, in der gespeichert
     *                       werden soll
     * @throws IOException    falls das Speichern aus irgendeinem
     *                       Grund fehlschlägt
     */
    public void speichereInDatei(String dateiname) throws IOException
    {
        Path zieldatei = Paths.get(dateiname).toAbsolutePath();
        ObjectOutputStream os = new ObjectOutputStream(
                                new FileOutputStream(
                                    zieldatei.toString()));
        os.writeObject(buch);
        os.close();
    }
}
```

Übung 14.52 Ändern Sie das Projekt *Netzwerk* aus Kapitel 11 so, dass die Daten in einer Datei gespeichert werden können. Verwenden Sie Objektserialisierung. Welche Klassen müssen Sie als serialisierbar deklarieren?

Übung 14.53 Was passiert, wenn Sie die Definition einer Klasse um, sagen wir, ein zusätzliches Datenfeld ändern und dann versuchen, serialisierte Objekte wieder einzulesen, die noch mit der alten Version der Klasse erzeugt wurden?

Zusammenfassung

Wenn zwei Objekte miteinander interagieren, dann kann es dabei aus den verschiedensten Gründen zu Problemen kommen. Beispielsweise:

- Der Programmierer eines Klienten hat ein falsches Verständnis vom Zustand oder von der Leistungsfähigkeit des Dienstleisters.
- Der Dienstleister kann eine Anfrage eines Klienten aufgrund von äußeren Umständen nicht erfüllen.
- Ein Klient wurde falsch programmiert und übergibt deshalb ungültige Parameterwerte an eine Methode des Dienstleisters.

Wenn etwas schiefgeht, dann wird das Programm entweder vorzeitig abbrechen oder es produziert falsche und unerwünschte Ergebnisse. Wir können viele dieser Probleme vermeiden, indem wir einen Exception-Mechanismus benutzen. Ein solcher Mechanismus gibt einem Dienstleister ein klar definiertes Mittel an die Hand, um einen Klienten über ein Problem zu informieren. Exceptions verhindern außerdem, dass ein Klient dieses Problem einfach ignoriert, und fordern Programmierer explizit dazu auf, einen alternativen Ausführungspfad für den Fehlerfall zu definieren.

Beim Entwickeln einer Klasse können **assert**-Anweisungen eingebaut werden, um interne Konsistenzprüfungen vorzunehmen. Diese Anweisungen werden normalerweise aus dem Produktionscode wieder entfernt.

Die Ein- und Ausgabe ist ein Bereich, in dem Exceptions häufiger auftreten. Das liegt vornehmlich daran, dass die Programmierer nur geringen (oder keinen) Einfluss auf die Umgebungen haben, in denen ihre Programme ausgeführt werden. Außerdem spiegelt es die Komplexität und Diversität dieser Umgebungen wider.

Die Java API unterstützt die Ein-/Ausgabe von Text- und Binärdaten mithilfe der Klassen für Reader, Writer und Streams. Neuere Java-Versionen haben die Pakete **java.nio** eingeführt, die einige der älteren **java.io**-Klassen ersetzt haben.

NEUE BEGRIFFE IN DIESEM KAPITEL

Exception, ungeprüfte Exception, geprüfte Exception, Exception-Handler, Zusicherung, Serialisierung





Zusammenfassung der Konzepte

- **Exception** Eine Exception ist ein Objekt, das Informationen über einen Programmfehler hält. Eine Exception wird ausgelöst, um zu signalisieren, dass ein Fehler aufgetreten ist.
- **Ungeprüfte Exception** Ungeprüfte Exceptions sind Exception-Typen, bei deren Verwendung der Compiler keine zusätzlichen Überprüfungen vornimmt.
- **Geprüfte Exception** Geprüfte Exceptions sind Exception-Typen, bei deren Verwendung der Compiler zusätzliche Überprüfungen durchführt und einfordert. Insbesondere erfordern geprüfte Exceptions in Java, dass **throws**-Klauseln und **try**-Blöcke angegeben werden müssen.
- **Exception-Handler** Ein Programmabschnitt, der Anweisungen schützt, in denen eine Exception ausgelöst werden kann, wird Exception-Handler genannt. Er definiert Anweisungen zur Meldung und/oder zum Wiederaufsetzen nach einer aufgetretenen Exception.
- **Zusicherung** Eine Tatsachenaussage, die bei normaler Programmausführung zutreffen sollte. Zu verwenden, um Annahmen zu formulieren und Programmierfehler leichter zu entdecken.
- **Serialisierung** Serialisierung erlaubt, ganze Objekte und Objekthierarchien in einer einzigen Operation zu lesen und zu schreiben. Jedes beteiligte Objekt muss dazu Instanz einer Klasse sein, die das Interface **Serializable** implementiert.

Übung 14.54 Ergänzen Sie das Projekt *Adressbuch* um die Fähigkeit, mehrere E-Mail-Adressen in einem **Kontakt**-Objekt zu speichern. Alle diese E-Mail-Adressen sollten gültige Schlüssel sein. Verwenden Sie auf jeder Stufe dieses Prozesses Zusicherungen und JUnit-Tests, um maximales Vertrauen in die endgültige Version zu schaffen.



KAPITEL

15

Entwurf von Anwendungen

Lernziele

Zentrale Konzepte in diesem Kapitel: Identifizieren von Klassen, Identifizieren von Klassen, CRC-Karten, Entwurfsmuster

Java-Konstrukte in diesem Kapitel: (In diesem Kapitel werden keine neuen Java-Konstrukte vorgestellt.)

In den vorangegangenen Kapiteln haben wir beschrieben, wie man gute Klassen schreibt. Wir haben diskutiert, wie man sie entwirft, wie man sie wartbar und robust macht und wie man sie miteinander interagieren lässt. All das ist wichtig, aber etwas sehr Wichtiges fehlt noch: das Finden der geeigneten Klassen.

In allen vorangegangenen Beispielen haben wir angenommen, dass wir mehr oder weniger wissen, welches die richtigen Klassen für unsere Problemlösungen sind. In einem realen Softwareprojekt kann das Finden der richtigen Klassen jedoch eines der schwierigsten Probleme sein. Diesen Aspekt der Softwareentwicklung wollen wir in diesem Kapitel näher betrachten.

Die frühen Phasen bei der Entwicklung eines Softwaresystems werden im Allgemeinen *Analyse und Entwurf* genannt. Wir analysieren das Problem, dann entwerfen wir eine Lösung. Der erste Entwurf wird dabei meist auf einer allgemeineren Ebene formuliert, als wir es in Kapitel 8 diskutiert haben. Wir werden zuerst überlegen, welche Klassen zur Problemlösung zu entwerfen sind und wie diese interagieren sollen. Sobald wir diesen Schritt abgeschlossen haben, können wir uns dem Entwurf der einzelnen Klassen und ihren Implementierungen zuwenden.

15.1 Analyse und Entwurf

Analyse und Entwurf von Softwaresystemen sind umfangreiche und komplexe Themengebiete. Ihre detaillierte Diskussion würde den Rahmen dieses Buches sprengen. Zahlreiche Methodiken sind in der Literatur beschrieben und werden in der Praxis eingesetzt. In diesem Kapitel wollen wir lediglich versuchen, einen Eindruck der mit diesen Tätigkeiten verbundenen Probleme zu geben.